CURSO REACT JS (Platzi)

Video 1

React.js nos ayuda a crear aplicaciones altamente escalables e interactivas, siendo multiplataforma, declarativo, basado en componentes.

Conocimientos básicos necesarios

- Programación básica
- HTML y CSS
- Javascript
- Git y GitHub
- NPM

Primeros pasos

- Clonar el repositorio https://github.com/platzi/curso-react-intro

```
git clone https://github.com/platzi/curso-react-intro.git
```

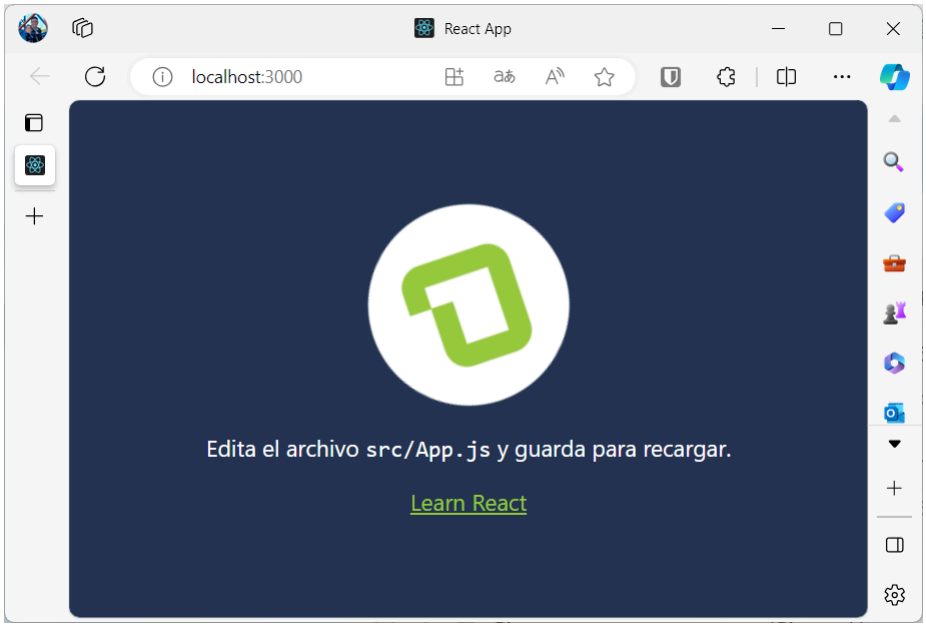
- Instalar las dependencias

```
npm i
```

- iniciar la aplicación

```
npm start
```

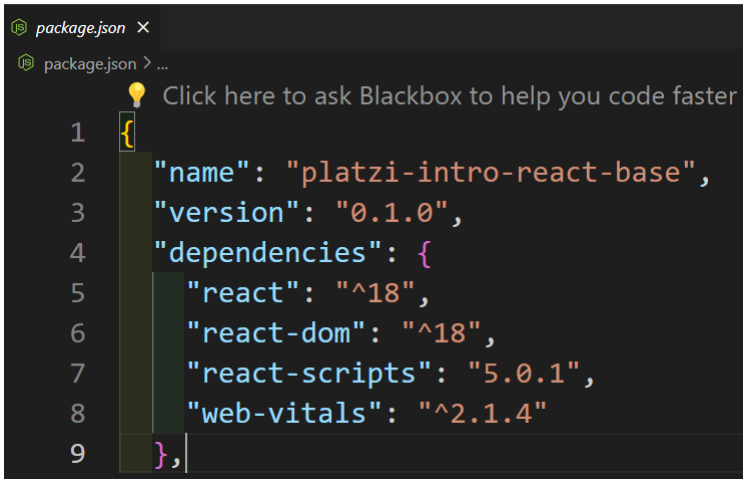
esto inicia un servidor de desarrollo y carga en el puerto 3000 la aplicación, deberíamos ver lo siguiente



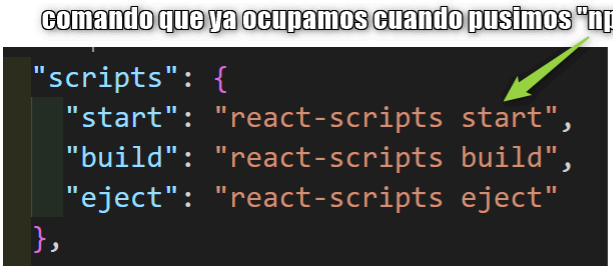
Video 2 - ¿Qué es un componente?

Inspección de los archivos del proyecto

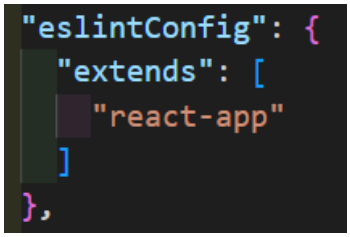
- Readme.md: **archivo con la explicación de la secuencia de comandos que hicimos en el primer video, también nos cuenta que el proyecto se creo con el comando** Create React App
- Package.json:
  - **archivo con el nombre del proyecto, su version, las dependencias**



  - **con los scripts que podemos ocupar en el proyecto** start **para ir probándolo mientras vamos desarrollando** build **para generar una versión para producción**



  - **configuración de esLint**(linter: una herramienta que examina el código buscando problemas, errores de sintaxis, e incluso ayudando a corregir malas prácticas) **uno de los linter de javascript más populares**



  - **configuración browsers tanto de producción como de desarrollo**



▶ Archivo Package.json

- .gitignore: **archivo que define los archivos que serán ignorados en los commits**

Análisis de carpetas del proyecto

- Public
  - Tiene un archivo **favicon.ico** que es el icono que mostrará el navegador
  - un archivo **index.html** que es el más importante del proyecto, dado que allí se renderiza react js
  - un archivo **manifest.json** que permite configurar la aplicación **react**, colores, si queremos que funcione offline, si va a ser **progressive web app**, etc..
  - un archivo **robots.txt** que permite definir a que se le puede hacer **webscraping**
- src
  - **index.js** es el archivo principal de react, que importa React(1), ReactDOM(2), estilos css(3), importamos el `spot por defecto`, el archivo **App.js** (el componente principal).

    En la línea 6 crea una constante `root` que usa **ReactDOM** para crear un **Root**, pasándole por parámetro el elemento ***div root*** del archivo **index.html**. En esa constante `root` creada usa el método `render` para renderizar el componente

```
1  import React from 'react';
2  import ReactDOM from 'react-dom/client';
3  import './index.css';
4  import App from './App';
5
6  const root = ReactDOM.createRoot(document.getElementById('root'));
7  root.render(<App />);
8
```

  - **app.js** es el archivo donde está el componente principal, que comienza importando un logo y sus estilos

```
1  import logo from './platzi.webp';
2  import './App.css';
```

    luego está la definición del componente, que se caracteriza por ser una función que siempre comienza en mayúsculas y que retorna una ***especie de html***, que

> en realidad es ***jsx*** `Es una extensión de la sintaxis de JavaScript popularizada por React.` que tiene una gran similitud con el **html** para ser legible y práctico para construir contenido de la página web con la programación que ofrece javascript.
>
> ```
> function App() {
>   return (
>     
>       <header className="App-header">
>         <img src={logo} className="App-logo" alt="logo" />
>         
>           Edita el archivo <code>src/App.js</code> y guarda para recargar.
>         
>         <a
>           className="App-link"
>           href="https://platzi.com/reactjs"
>           target="_blank"
>           rel="noopener noreferrer"
>         >
>           Learn React
>         </a>
>       </header>
>     
>   );
> }
>
> export default App;
> ```
>
> a las **"etiquetas html"** que vemos en **jsx** se las conoce como **elementos de react**, que luego de que se transpile se transformarán en etiquetas html.
>
> Se puede observar que el archivo finaliza con la línea `export default App;` esto se conoce por **export default** y si bien es muy práctico a la hora de importar dado que solo tendremos que hacer `import App from './App';` no nos ofrece controles de error como si lo hacen los **export nombrados**

Probamos crear un Componente: TodoItem

Creamos una función que declara al componente, le ponemos de nombre **TodoItem(recordar que deben estar en mayúsculas)**, `tenemos que tener en claro que los componentes tienen que retornar solo un elemento(es decir una etiqueta que contenga a todo el retorno)` en nuestro caso será un elemento **li**(etiqueta) que contiene el contenido del componente.

```
function TodoItem() {
  return (
    <li>
      <span>V</span>
      <p>Elemento de prueba</p>
      <span>X</span>
    </li>
  );
}
```

- para poder ver el componente renderizado es necesario llamarlo como una etiqueta, lo colocaremos dentro del componente **App**

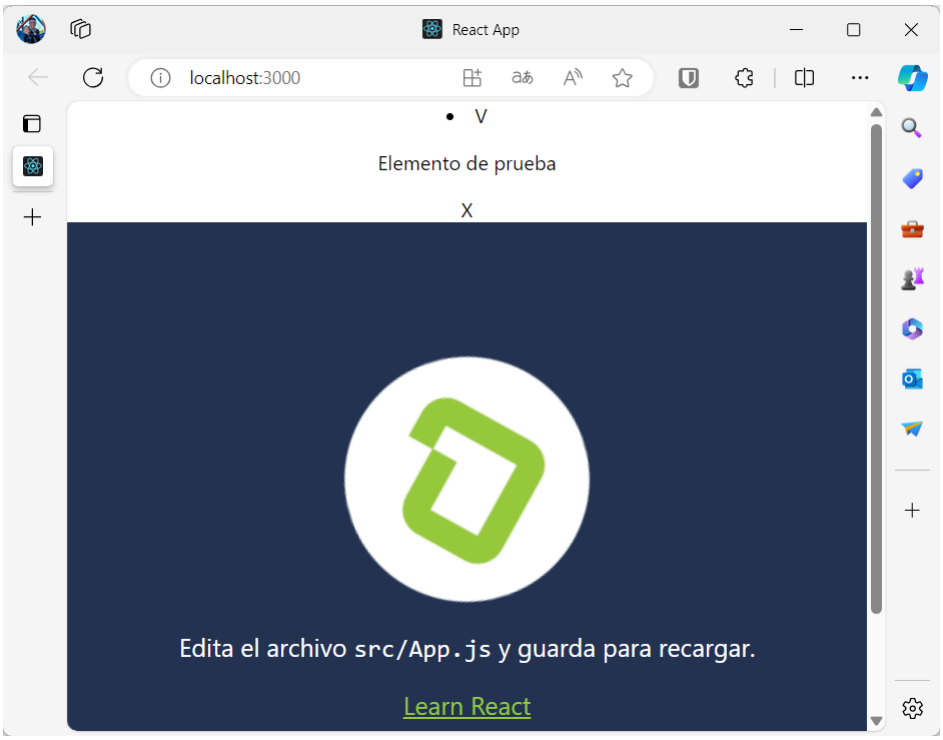
```
function App() {
  return (
    <div className="App">

      <TodoItem />
```

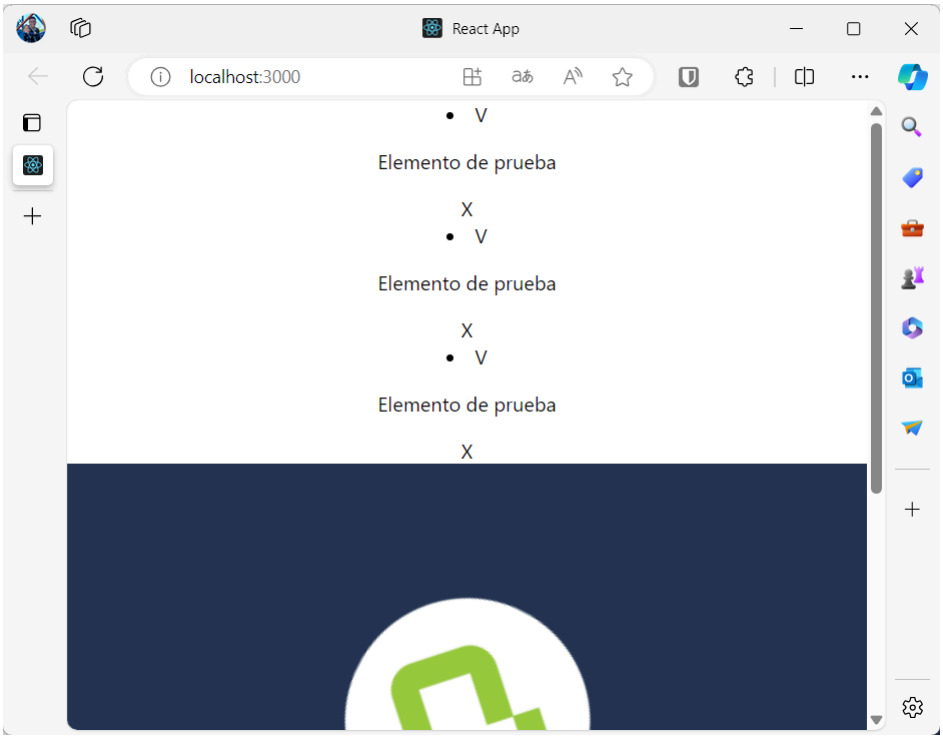
- nos vamos a la terminal y ejecutamos

```
npm start
```

y vamos a poder ver en el navegador que ya aparece nuestro componente **TodoItem** al comienzo de la **App**



si probamos repetir la llamada al componente veremos como aparece varias veces.



Creamos nuestro segundo componente de prueba: TodoCount

En este caso usaremos un elemento **h1**(etiqueta) como elemento contenedor del contenido del componente.

```
function TodoCount(){
  return (
    <h1>
      Completaste
      3 de 5 TODOs
    </h1>
  );
}
```

Probamos las PROPs para enviar parámetros al componente

Las **props** es un recurso que nos permite enviarle información al componente, funcionan exactamente igual que la recepción de parámetros de cualquier función **javascript**, pudiendo enviarse:

- de manera **"estructurada"** como un `objeto` que contiene `propiedades`

```
function TodoCount(props){
  return (
    <h1>
      Completaste
      {props.completed} de {props.total} TODOs
    </h1>
  );
}
```

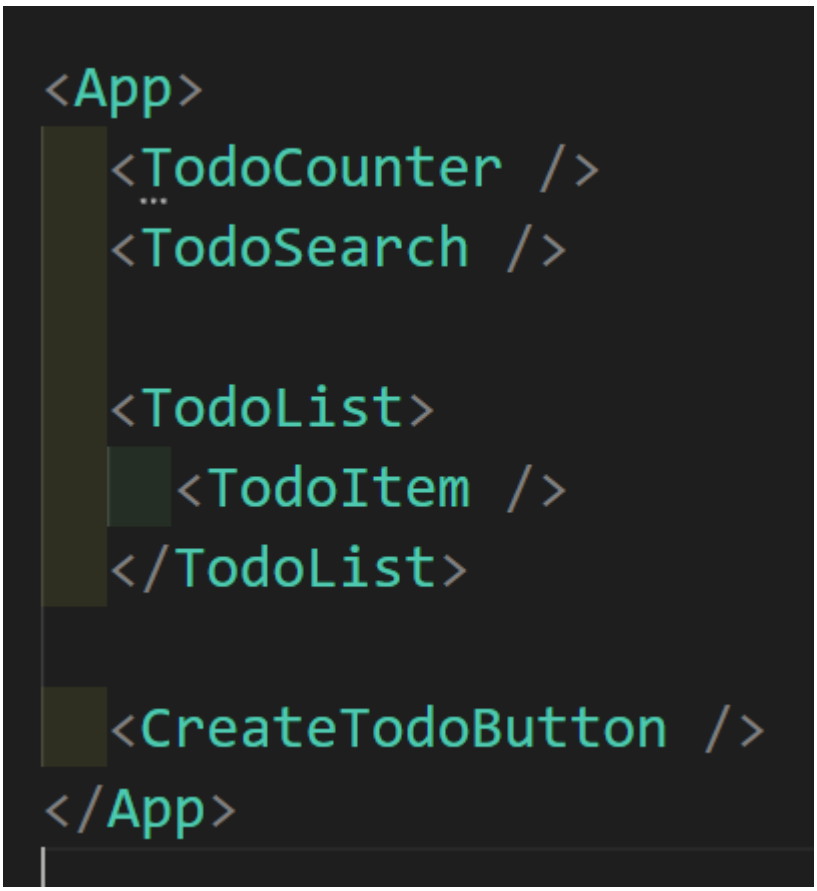
- o **"descriptiva"** enviándolas entre llaves y nombrándolas separadas por comas

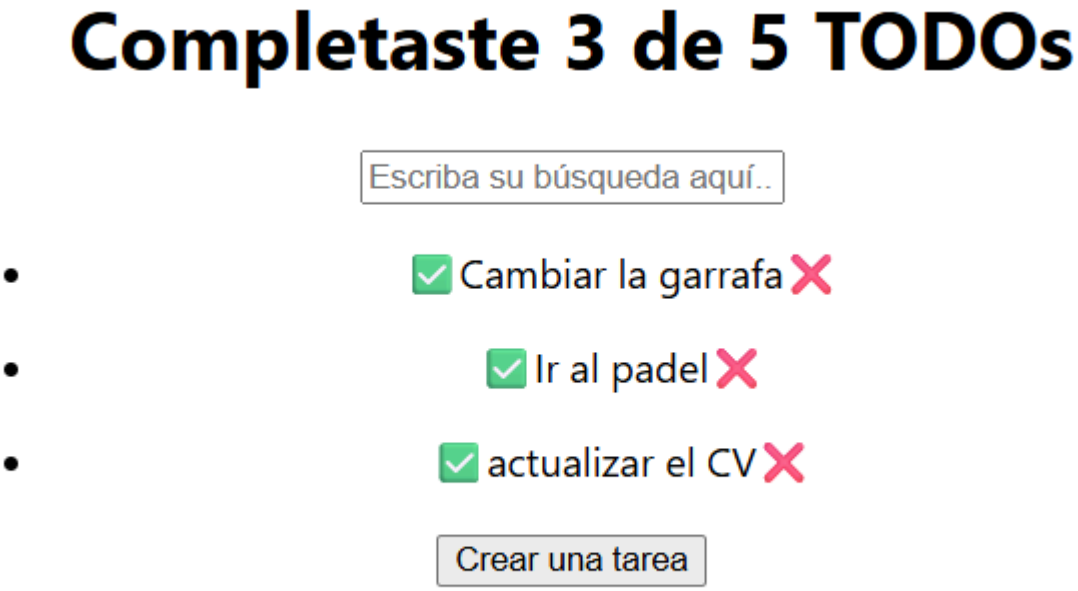
```
function TodoCount({completed, total}) {
  return (
    <h1>
      Completaste
      {completed} de {total} TODOs
    </h1>
  );
}
```

- Para enviarlas al momento de llamar al componente tenemos que escribirlas como si fuesen **atributos html**, pero sin perder de vista que en realidad son **propiedades**.

```
<TodoCount completed={3}  total={5}/>
```

Video 3 - Componentes de TodoMachine





Creamos archivos para cada uno de los componentes

- Creamos **TodoCounter.js** (en la carpeta **src**)

```
TodoCounter.js U
TodoCounter.js > TodoCounter
Click here to ask Blackbox to help you code faster
function TodoCounter({completed, total}) {
    return (
        <h1>
            Completaste {completed} de {total} TODOs
        </h1>
    );
}

export {TodoCounter};
```

`Usamos export nombrados: que controlan en la importación que no haya errores en la importación en los nombres de los componentes` Ej: **export { TodoCounter }**

`y cuando lo querramos utilizar tendremos que importarlo de la misma manera, que tiene un control extra que evita errores en la denominación` Ej: **import {TodoCounter} from './TodoCounter';**

- Creamos **TodoSearch** (en la carpeta **src**)

```
TodoSearch.js U
src > TodoSearch.js > ...
Click here to ask Blackbox to help you code faster
function TodoSearch() {
    return (
        <input placeholder="Escriba su búsqueda..." />
    );
}

export {TodoSearch};
```

- Creamos **TodoList** (en la carpeta **src**)

```
TodoList U
src > TodoList > ...
Click here to ask Blackbox to help yo
function TodoList(props) {
    return (
        <ul>
            {props.children}
        </ul>
    );
}

export {TodoList};
```

- Ubicamos **TodoItem** en un archivo (en la carpeta **src**)

```
TodoItem.js U
src > TodoItem.js > ...
Click here to ask Blackbox to help you co
function TodoItem() {
  return (
    <li>
      <span>V</span>
      <p>Elemento de prueba</p>
      <span>X</span>
    </li>
  );
}

export { TodoItem };
```

- Creamos **CreateTodoButton** (en la carpeta **src**)

```
src > CreateTodoButton.js > ...
Click here to ask Blackbox to help you code faster
function CreateTodoButton() {
    return (
        <>
            <input type="button" value="Crear una tarea" />
        </>
    );
}

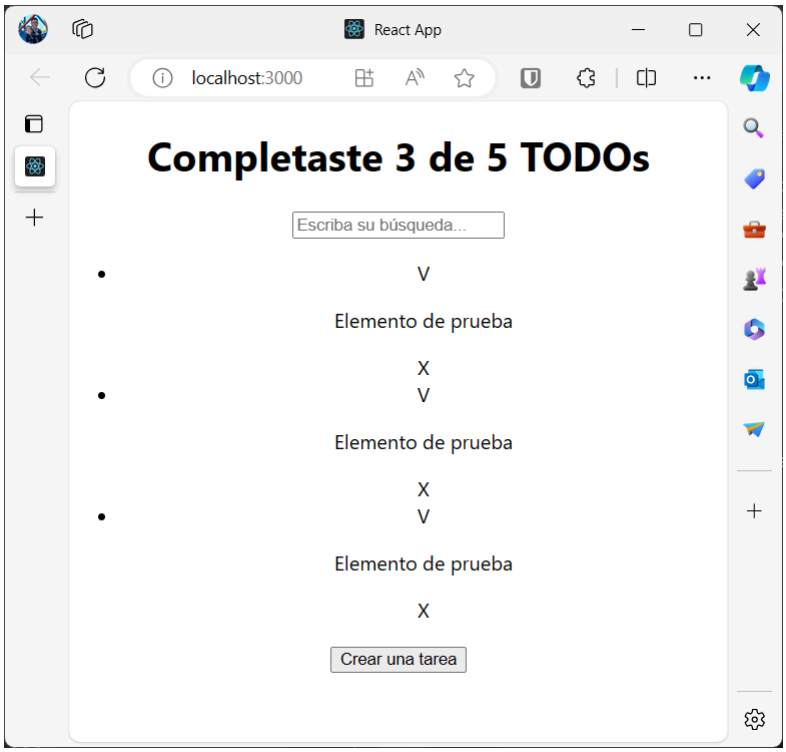
export {CreateTodoButton};
```

- importamos todos los archivos en el archivo principal **App.js**

```
import { TodoCounter } from './TodoCounter';
import { TodoSearch } from './TodoSearch';
import { TodoList } from './TodoList';
import { TodoItem } from './TodoItem';
import { CreateTodoButton } from './CreateTodoButton';
```

`aclaración: el comentariado en **react** se hace con {/*  */}`
`(llave+barra+asterísco)`

- probamos correr la aplicación con el comando **npm start** y deberíamos ver los siguiente



- el archivo **App.js** nos quedó así (como hicimos dinámico al componente TodoCounter le tuvimos que pasar por **props** los valores que despliega)

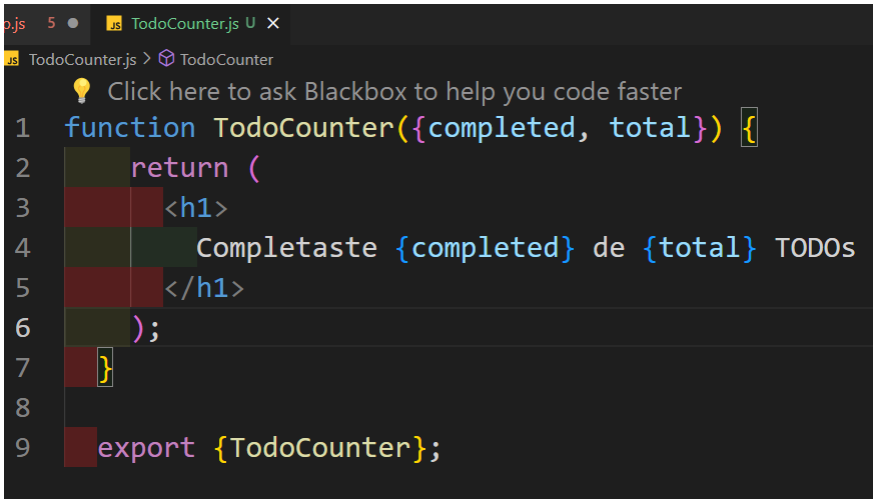
```
src > App.js > App
Click here to ask Blackbox to help you code faster
1  import logo from './platzi.webp';    'logo' is defined
2  import { TodoCounter } from './TodoCounter';
3  import { TodoSearch } from './TodoSearch';
4  import { TodoList } from './TodoList';
5  import { TodoItem } from './TodoItem';
6  import { CreateTodoButton } from './CreateTodoButton';
7  import './App.css';
8
9  function App() {
10   return (
11     <div className="App">
12       <TodoCounter completed={3} total={5} />
13       <TodoSearch />
14
15       <TodoList>
16         <TodoItem />
17         <TodoItem />
18         <TodoItem />
19       </TodoList>
20
21       <CreateTodoButton />
22     </div>
23   );
24 }
25
26 export default App;
27
```

Video 4 - ¿Como se comunican los componentes? Props y atributos

Las **props** son variables que declaramos en nuestro componente como si fueran parámetros de una función, podemos declarar una unica (Prop) o declarar varias Props utilizando la {Desestructuración}. Para luego posteriormente darles un valor como si fueran argumentos, pero utilizando otro tipo de sintaxis, que es muy parecida a la de los atributos de los elementos HTML, con la diferencia que le otorgamos el valor entre {llaves}. ...

```
<TodoCount completed={3}  total={5}/>
```

Como ya lo hicimos en **TodoCounter**



Children

En cuanto a la propiedad **"props.children"** esta hace referencia a que **React** transforma automáticamente a todo lo que se encuentre dentro del componente padre en un elemento `"children"` , es decir todo lo que se encuentre dentro de una etiqueta de apertura `<Component>` y una etiqueta de cierre `</Component>` para **React** será `"children"`. ..

<React.Fragment> o su formato abreviado <> </>

Por otro lado existe `<React.Fragment>` o su sintaxis más corta " <> </> " que es un contenedor invisible que tiene como objetivo envolver todos los componentes JSX dentro de un contenedor padre, esto es debido a que React necesita envolver esos componentes para renderizarlos, y asi no crear nodos innecesarios en el DOM.

- Probamos crear un array de Tareas(TODOs)

```
const listaTareas = [
    { text: 'Cambiar la garrafa', completed:false},
    { text: 'Ir al padel', completed:false},
    { text: 'actualizar el CV', completed:false}
]
```

- luego utilizamos ese array para renderizar componentes utilizando sus elementos, recorriéndolos con **map**, este método nos permite llamar a una función flecha para cada elemento dentro del array, allí crearemos un componente **TodoItem** por cada elemento y enviaremos las propiedades al componente, ***Aclaración: al iterar un array React nos pide***

que enviemos una propiedad `key` para identificar de manera única a cada elemento del array

```
<TodoList>
  {listaTareas.map(tarea=>(
    <TodoItem
      key={tarea.text}
      text={tarea.text}
      completed= {tarea.completed}
    />
  ))}
</TodoList>
```

Video 5 - Estilos CSS en React

Hay 2 formas de agregar **estilos CSS** en **React**:

- `Estilos en línea`: es una forma especial de escritura de **CSS** que tiene **JSX** donde a los elementos se les escribe **CSS** en la propiedad ***style*** pero escribiendolo con 3 particularidades:
    - se escriben dentro de 2 llaves como si fuesen objetos
    - se escriben en ***camelcase*** eliminando los guiones bajos
    - se separan con comas porque son propiedades de un objeto

```
<h1 style={{
  fontSize: '24px',
  textAlign: 'center',
  margin: 0,
  padding: '48px'
}}>
```

    - otra posibilidad es definir los estilos en un objeto y asignarlos en la propiedad **style** del elemento **React**

```
const estilos = {
  backgroundColor: 'red'
}

function TodoCounter({ total, completed }) {
  return (
    <h1 style={estilos}>
      Has completado {completed} de {total}
      TODOS
    </h1>
  );
}

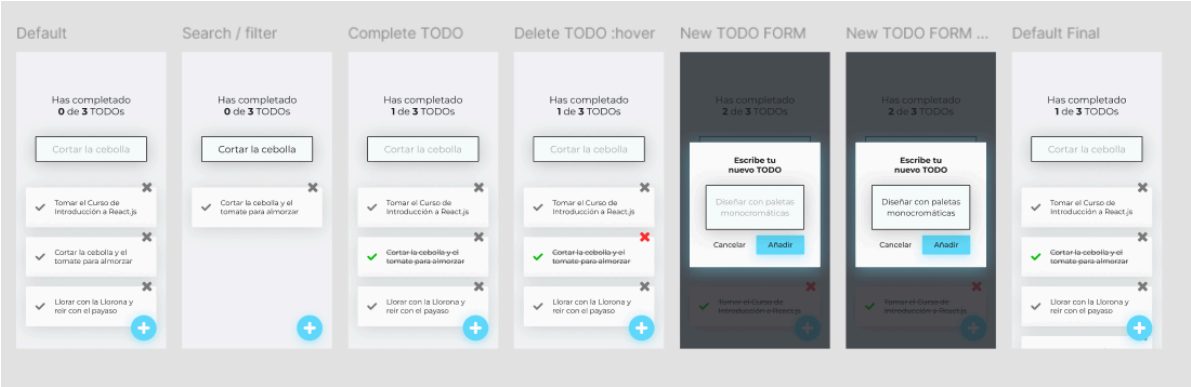
export { TodoCounter };
```

- `Hojas de estilos`: se escriben archivos **CSS** que por una cuestión de organización se les dá el mismo nombre que el componente que formatean y se los importa en el archivo del **componente**.

```
import './App.css';
```

▶ Archivo TodoList.css

▶ Archivo TodoSearch.css
▶ Archivo index.css
▶ Archivo TodoCounter.css
▶ Archivo TodoItem.css



Video 6 - Eventos en React

Hacer que una aplicación React reaccione a las acciones de quien la usa empieza por **escuchar eventos en React JS** con atributos como `onClick` y `onChange`. Sin esa capa, tu interfaz luce bien pero permanece inerte: los botones no responden, los inputs no filtran y la lista de tareas no se actualiza.

En este video se vé cómo capturar clics, lecturas de teclado y datos del input para preparar el terreno hacia el manejo de estado.

¿Cómo se diferencia onClick en React de onclick en HTML?

En HTML clásico, `onclick` va en minúsculas y recibe un *string* con código JavaScript. En **React**, la sintaxis cambia: se escribe en ***camelCase*** y recibe una función dentro de llaves.

React toma cualquier atributo que empiece con `on` (como `onClick`, `onChange`, `onScroll`) y lo traduce internamente a un `addEventListener` de JavaScript. Por eso, si inspeccionas el botón en el DOM, no verás un atributo `onclick` visible: el escuchador vive en los ***event listeners*** del navegador.

> **¿Por qué onClick va en camelCase en React?** Porque React procesa cualquier prop que empiece con `on` y la convierte en un `addEventListener`. La convención *camelCase* permite distinguir eventos de atributos HTML normales.

¿Por qué hay que envolver el código en una función?

No puedes pasar directamente un `console.log('le diste clic')` dentro de las llaves. Si lo haces así, el código se ejecuta cuando el componente se renderiza, no cuando el usuario interactúa.

La solución es encapsular la lógica en una ***arrow function*** para entregársela a **React** y que él decida cuándo ejecutarla:

```
<button className="create-todo-button"
    onClick={() => { console.log('le diste clic'); }}>
    Crear una tarea
</button>
```

Así, React solo dispara la función cuando el usuario hace clic real sobre el botón.

¿Qué información trae el parámetro event en React?

Cada función de evento recibe un parámetro especial: el ***event***. Al imprimirlo en consola aparece como un `SyntheticBaseEvent`, una versión que **React** envuelve sobre el evento nativo del navegador.

Entre sus propiedades destacan:

- **type**: el tipo de evento, por ejemplo `click` o `change`.
- **target**: el nodo HTML desde el cual se disparó el evento.
- Información adicional sobre coordenadas, teclas presionadas o el estado del DOM.

La propiedad `target` es de las más usadas porque te da acceso directo al **elemento HTML**, como si lo hubieras seleccionado con manipulación tradicional del **DOM** . Desde ahí puedes leer clases, contenido interno o cualquier atributo.

¿Cómo capturar el texto de un input con onChange?

El evento `onChange` es la pieza clave para **inputs** y campos de búsqueda. Funciona igual que `onClick`: recibe una función que **React** ejecuta cada vez que el contenido del input cambia.

En el componente `TodoSearch`, el código queda así:

```
<input placeholder="Cortar cebolla"
       onChange={
           (event) => {
               console.log('Escribiste en el TodoSearch');
               console.log(event.target.value);
           }
   }/>
```

La propiedad `event.target.value` devuelve el texto actual del input. Si el usuario escribe "cortar", **React** dispara un evento por cada letra: primero `c`, luego `co`, después `cor`, hasta `cortar`. Incluso detecta cuando se borra un carácter .

> **¿Qué hace event.target.value en React?** Devuelve el valor actual del elemento que disparó el evento. En un input, equivale al texto que el usuario tiene escrito en ese momento exacto.

¿Para qué sirve capturar cada cambio en tiempo real?

Este comportamiento de captura por tecla es lo que permite construir buscadores en vivo. Cada pulsación se traduce en información que tu aplicación puede usar para filtrar la lista de tareas sin esperar a que el usuario presione **Enter** o un botón de buscar.

Es el mismo principio que aplican ***autocompletadores*** y filtros instantáneos en interfaces modernas.

¿Qué eventos comunes puedes escuchar en React?

React soporta prácticamente cualquier evento del DOM. Algunos de los más usados:

- `onClick`: clics sobre botones, enlaces o cualquier elemento interactivo.
- `onChange`: cambios en inputs, selects y textareas.
- `onSubmit`: envío de formularios.
- `onScroll`: desplazamiento dentro de un contenedor.
- `onMouseEnter` y `onMouseLeave`: paso del cursor sobre un elemento.

Todos siguen la misma regla: ***camelCase***, función dentro de llaves y acceso al objeto `event` como parámetro.

Con esto ya puedes detectar interacciones, pero todavía falta el siguiente paso: comunicar esa información entre componentes para que, por ejemplo, la lista de tareas se filtre según lo que se escribe en el buscador. Esa pieza se llama estado y es lo que abre la puerta a aplicaciones verdaderamente reactivas.

Video 7 ¿Qué es el estado?

Guardar las interacciones de los usuarios en una aplicación de **React** requiere algo más que un *console.log*. Aquí entra **useState**, la herramienta que te permite almacenar y actualizar información cada vez que alguien **escribe**, hace **clic** o **interactúa** con tu interfaz, y que además dispara un re-renderizado automático para reflejar esos cambios.

¿Qué es useState y por qué se importa desde React?

UseState es una función de **React** que crea un ***estado*** dentro de un componente. Al llamarla devuelve un array con dos elementos: el valor actual del estado y una función para actualizarlo.

Para usarla necesitas importar **React** explícitamente. Quizá notaste que en componentes anteriores no lo hacíamos, y eso pasa porque **Create React App** incluye un plugin de **Babel** que importa **React** por ti. Si quieres prevenir errores futuros, importa **React** en cada componente.

> **¿Qué hace useState en React?** Crea un **estado** dentro de un **componente** y **devuelve un array** con dos elementos: el ***valor actual*** y una ***función actualizadora***. Cada vez que llamas a esa función, **React** re-renderiza el componente.

¿Por qué useState devuelve un array con dos elementos?

El estado en **React** es **inmutable**, así que no puedes reasignarlo con un *signo igual*. Necesitas una función actualizadora controlada por **React** para que maneje la inmutabilidad y el historial de estados sin romper nada.

La convención de nombres es clara:

- El primer elemento es el estado, por ejemplo `searchValue`.
- El segundo es el actualizador, con el prefijo ***set*** y ***camelCase***, por ejemplo `setSearchValue`.
- Si tu estado se llamara `patito`, su actualizador sería `setPatito`.

UseState también acepta un valor inicial. En el caso del input de búsqueda, lo correcto es un string vacío para que aparezca el ***placeholder*** hasta que el usuario empiece a escribir.

¿Cómo conectar un input con el estado de React?

La propiedad `value` del input se enlaza directamente al estado. Así, lo que el usuario vea siempre será el reflejo del valor almacenado en **React**, no del comportamiento por defecto del HTML.

```
const [searchValue, setSearchValue] = React.useState('');

<input value={searchValue}
        onChange={
            (event) => {
                    setSearchValue(event.target.value);
            }
    } />
```

Si solo defines `value={searchValue}` sin actualizar el estado, el input se queda congelado: aunque el usuario teclee, no aparece nada. La razón es que el **value** está atado al estado, y si el estado nunca cambia, el input tampoco.

La solución es llamar a `setSearchValue(event.target.value)` dentro del ***onChange***. Con eso, cada tecla actualiza el **estado** y el **estado** actualiza el **input**.

¿Cómo confirmar que el estado realmente guarda la información?

Un truco útil es colocar un `console.log` antes del *return* del componente, no dentro del evento. De esa forma compruebas que el estado está accesible en todo el componente, no solo dentro del ***onChange***.

```
console.log(`Los usuarios buscan todos de ${searchValue}`);
```

Escribimos algo y veremos en **consola** el log actualizándose letra por letra. Eso confirma que el dato vive en el estado y puede compartirse con otros **componentes**.

¿Por qué no basta con una variable normal de JavaScript?

Aquí está el detalle que mucha gente pasa por alto. Una variable común cambia su valor, pero **no avisa a React** de que la interfaz debe volver a renderizarse.

Imagina un componente `TodoClick` con una variable `n = 0` y un botón que hace `n = n + 1`. Le das clic una vez, dos veces, tres veces, y aunque la variable internamente sí suma, la interfaz sigue mostrando cero. **React** no recibió la señal de re-render.

Cuando reemplazas esa variable por un estado con `React.useState(0)` y llamas a `setState(state + 1)` en el ***onClick***, ocurre la magia: el estado sube y la interfaz se actualiza en sincronía.

> **¿Por qué un estado provoca re-render y una variable no?** Porque el actualizador del **estado** le notifica a **React** que debe volver a renderizar el componente. Una variable de JavaScript cambia silenciosamente sin avisar al motor de render.

¿Qué es un re-renderizado en React?

Es el proceso por el cual **React** vuelve a procesar un componente después de un cambio de **estado**. La ***"fábrica de render"*** recibe el componente con su nuevo estado, genera el **HTML** y **JavaScript** actualizados, y los envía al navegador.

El flujo se ve así:

1. El usuario dispara un **evento**, por ejemplo un **clic**.
2. El **estado** cambia gracias a la ***función actualizadora***.
3. **React** detecta el cambio y manda el componente afectado a la ***fábrica de render***.
4. La fábrica genera el nuevo **HTML** y lo refleja en el navegador.

¿Qué es el Virtual DOM y por qué importa?

React no manipula el **DOM** real directamente cada vez que algo cambia. En su lugar, mantiene una copia llamada *virtual DOM*. Cuando hay un re-render, los cambios se aplican primero a esa copia, se comparan con el DOM real y solo se actualizan las partes que realmente cambiaron.

Esto evita que toda la aplicación se redibuje en cada interacción. Solo los componentes con estado modificado pasan por la ***fábrica***, lo que mantiene a **React** rápido incluso en aplicaciones grandes.

Contando TODOs

> Crear interactividad y cohesión entre componentes es una habilidad esencial en el desarrollo moderno con **ReactJS**, una biblioteca de **JavaScript** para construir interfaces de usuario. Esto incluye desde manipular el estado de los componentes hasta hacer que estos se comuniquen efectivamente. En esta exploración, descubriremos cómo se pueden vincular componentes padres e hijos en **React**, así como la generación de estados derivados que permiten crear cálculos dinámicos y precisos basados en estados previos, utilizando como ejemplo practico la implementación de un contador en una aplicación de tareas o "todos".

¿Cómo pueden comunicarse estados entre componentes padres e hijos en React?

Para lograr la comunicación de estados en **React**, se debe tomar en cuenta que no es posible compartir **estados** directamente de un componente hijo a un componente padre. Lo que sí es posible es realizar esta comunicación **de padres a hijos**. Esto implica que el estado debe ser definido en el **componente padre** para después ser pasado al **componente hijo** a través de **props**, respetando una jerarquía clara donde el flujo de datos va en una sola dirección descendente.

¿Es posible crear estados derivados a partir de los estados existentes?

Crear ***estados derivados*** es un método poderoso para realizar ***cálculos basados en estados anteriores***. Por ejemplo, si queremos llevar un registro del número de tareas completadas y el total de tareas, necesitamos calcular estos valores basándonos en el estado actual de la lista de tareas. Esto se consigue mediante la utilización de métodos de manipulación de arrays en **JavaScript**, tales como `filter` o `map`, que ***nos permiten derivar nueva información sin alterar el estado original***.

Implementando el contador de tareas

Para que el componente `TodoCounter` muestre información precisa en vez de números arbitrarios, se requiere de la manipulación del estado de tareas o "todos". Primero, se traslada el estado relacionado con la búsqueda de tareas del componente `TodoSearch` al componente padre, en este caso `App`. Luego, se utilizan las **props** para pasar tanto el estado como su actualizador (`setSearchValue`) de nuevo al `TodoSearch` y a cualquier otro componente que lo requiera.

▶ TodoSearch.js con recepción de estados por Props

El proceso se completa creando dos nuevas variables: `completedTodos` y `totalTodos`. La primera representa el número de tareas completadas, calculado a partir del estado de tareas existente, y la segunda simplemente refleja la longitud total del array de tareas. Estos cálculos nos permiten obtener valores dinámicos que se actualizan en respuesta a los cambios en el estado de las tareas.

Desarrollo práctico y continuo

Con estas técnicas, se logra no solo compartir y actualizar estados dentro de una aplicación **React**, sino también derivar nueva información y funcionalidades a partir de ellos. Esta clase sienta las bases para que en futuras lecciones se pueda abordar la búsqueda de tareas de forma dinámica, transformando las interacciones del usuario en acciones concretas que modifican la **visualización de la lista de tareas**.

> **ReactJS** es más que una simple biblioteca; *es un ecosistema que proporciona a los desarrolladores las herramientas necesarias para crear aplicaciones interactivas y cohesivas*. Siguiendo estos pasos y practicando con los conceptos, te encontrarás bien equipado para manejar **estados** y **props** de manera efectiva en tus futuros proyectos.

Filtrar TODOs con estados derivados en React

Construir un buscador funcional en **React** requiere combinar **estados**, **props** y **estados derivados** para filtrar información en tiempo real. Aquí aprenderás a implementar un filtro de búsqueda dentro de una aplicación de **TODOs**, manejando coincidencias entre mayúsculas y minúsculas y conectando varios componentes de forma reactiva.

Esta guía te muestra paso a paso cómo derivar un nuevo array a partir del estado original, cómo usar los métodos `filter` e `includes`, y cómo refactorizar tu código para que sea más legible y declarativo.

¿Cómo funciona un estado derivado para buscar TODOs?

Un **estado derivado** es un valor que se calcula a partir de otro **estado** existente. En lugar de guardar la lista filtrada como un estado independiente, la generas cada vez que cambia el estado original.

En este caso parto de dos estados que ya existen en el componente: la lista completa de **TODOs** y el `searchValue` que el usuario escribe en el buscador. A partir de ellos creo `searchedTodos`, que filtra la lista cada vez que el input cambia.

```
 const searchedTodos = todos.filter(todo =>

todo.text.toLowerCase().includes(searchValue.toLowerCase()));
```

El método `filter` recorre cada elemento del array y devuelve uno nuevo solo con los que pasen la validación. La validación aquí usa `includes`, un método de los **strings** que pregunta si un texto contiene a otro en cualquier parte.

> **¿Qué hace el método filter en JavaScript?** Recorre un array y devuelve uno nuevo con los elementos que cumplan una condición que tú defines dentro de una función. Si retornas `true`, el elemento entra; si retornas `false`, se descarta.

¿Por qué mi buscador no muestra resultados?

Un error muy común cuando empiezas con arrow functions y llaves es olvidar el `return`. Si tu función filter usa llaves `{}` pero no retorna nada, el resultado siempre será `undefined`, que se interpreta como falso, y el array filtrado quedará vacío.

Tienes dos formas de resolverlo:

- Agregar explícitamente la palabra `return` antes de la validación.
- Reemplazar las llaves por paréntesis `()` para que la **arrow function** retorne implícitamente.

Ambas opciones funcionan igual. Lo importante es entender que `filter` necesita un valor booleano por cada iteración para decidir si el elemento se queda o no.

```
//retorno implicito
const searchedTodos = todos.filter(todo =>
  todo.text.toLowerCase().includes(searchValue.toLowerCase())
);

//retorno explicito
const searchedTodos = todos.filter((todo) => {
  const todoText = todo.text.toLowerCase();
  const searchText = searchValue.toLowerCase();
  return todoText.includes(searchText);
});
```

¿Dónde renderizo el estado derivado?

El componente `TodoList` ya no debe renderizar los **TODOs** por defecto ni el array original sin filtrar. Debe renderizar `searchedTodos`, que es la versión filtrada y reactiva de tu lista.

Esto significa que cuando el usuario escribe en el buscador, el estado `searchValue` cambia, lo que dispara un nuevo cálculo de `searchedTodos`, y la lista visible se actualiza automáticamente. Toda la cadena reactiva fluye sola.

¿Cómo hago un buscador insensible a mayúsculas y minúsculas?

El método `includes` es estricto: si buscas "la" en minúsculas, no encontrará "LALALA" en mayúsculas. Para muchos buscadores eso es un problema, porque el usuario espera encontrar coincidencias sin importar cómo escribió.

La solución es normalizar ambos textos a minúsculas antes de comparar, usando el método `toLowerCase`:

```
const searchedTodos = todos.filter((todo) =>
                        {
                            const todoText = todo.text.toLowerCase();
                            const searchText = searchValue.toLowerCase();
                            return todoText.includes(searchText);
                        });
```

Esta transformación solo ocurre dentro de la validación. El usuario sigue viendo los textos originales con sus mayúsculas y minúsculas intactas.

> **¿Qué hace toLowerCase en JavaScript?** Convierte un string completo a minúsculas. Es útil para comparaciones donde no quieres que la diferencia entre mayúsculas y minúsculas afecte el resultado.

¿Por qué conviene separar la lógica en variables?

Escribir toda la validación en una sola línea funciona, pero perjudica la legibilidad. Separar `todoText` y `searchText` en variables independientes hace que el código sea más declarativo y fácil de leer cuando vuelvas a él semanas después.

El JavaScript moderno permite encadenar muchos métodos, pero declarar variables intermedias con nombres claros es una buena práctica que tu yo del futuro va a agradecer.

¿Qué herramientas de React combinas en este buscador?

Este ejercicio integra varios conceptos que ya viste en clases anteriores y los conecta en una sola funcionalidad real:

- **Estado local con useState** para guardar la lista de **TODOs** y el valor del buscador.
- **Estados derivados** que se recalculan en cada render a partir de otros **estados**.
- **Comunicación entre componentes** pasando estados y actualizadores como **props** desde el componente padre.
- **Métodos de arrays y strings** como `filter` e `includes` para transformar datos sin mutarlos.

La aplicación se siente viva porque cada interacción del usuario, por pequeña que sea, dispara una cadena reactiva que actualiza varios componentes a la vez.

¿Cuál es el siguiente reto?

El reto es usar el actualizador `setTodos` para completar o eliminar **TODOs** de forma independiente. Necesitas pasar ese actualizador a cada `TodoItem` y modificar el estado original cuando el usuario haga clic en el botón de completar o eliminar.

Ese cambio debe propagarse al `TodoCounter`, al `TodoSearch` y a la `TodoList` automáticamente, gracias a la reactividad que ya construiste.

✅Completar y ❌eliminar to-dos con eventos React

Marcar una tarea como completada o eliminarla con un clic es lo que separa una *lista estática* de una **aplicación viva**. Acá vas a aprender cómo manejar eventos en componentes hijos de **React** usando ***funciones encapsuladas***, manipulación de arrays y el actualizador de estado **setTodos** para que tu ***to-do app*** reaccione a cada interacción del usuario.

Cómo disparar un evento onClick desde un componente hijo en React

`El primer paso para completar un *to-do* es decidir dónde vive la lógica. Y aquí viene la decisión clave: el componente TodoItem no debería preocuparse por filtrar arrays ni recalcular estados.`

En lugar de pasarle el actualizador global `setTodos` a cada *to-do item*, le pasamos una propiedad más específica llamada `onComplete`. Esto se siente como un evento natural: cuando el usuario hace clic en el ícono de check, el componente solo dispara `props.onComplete` y listo. Toda la lógica pesada se queda en el componente padre.

> **¿Por qué no pasar setTodos directamente al componente hijo?** Porque obligarías a `TodoItem` a conocer la estructura completa del estado y a hacer cálculos que no le corresponden. Pasarle una función específica como `completeTodo` mantiene cada componente enfocado en una sola responsabilidad.

En el `span` del ícono de completar agregamos el evento `onClick`, y dentro llamamos a `props.onComplete`. El componente hijo queda limpio: recibe una función y la ejecuta cuando corresponde.

Cómo crear la función completeTodo con setTodos y manipulación de arrays

En el componente padre creamos la función `completeTodo` que recibe un texto como identificador único. Como ya definimos que el texto del *to-do* es la *key* única en el render del *array*, lo usamos también para encontrar cuál modificar.

La lógica funciona así:

- Crear una copia del estado actual usando el operador *spread* (`[...todos]`). Esto evita mutar el estado original, algo que React no permite.
- Encontrar el índice del *to-do* que queremos modificar con el método `findIndex`, comparando `todo.text` con el texto recibido.
- Acceder al objeto en esa posición del nuevo *array* y cambiar su propiedad `completed` a `true`.
- Llamar a `setTodos(newTodos)` para actualizar el estado.

```
const completeTodo = (text) => {
        const newTodos = [...todos];
        const todoIndex = newTodos.findIndex((todo) => todo.text === text );
        newTodos[todoIndex].completed = true;
        setTodos(newTodos); };
```

Esta lógica la escribes una sola vez y la reutilizas para todos tus *to-dos*. ***El componente hijo no se entera de nada.***

Por qué aparece el error too many re renders en React

Al pasar la función al componente hijo, hay una trampa que casi todo desarrollador cae al menos una vez. Si escribes `onComplete={props.onComplete(todo.text)}` con paréntesis, **React** ejecuta la función en cada render, lo que actualiza el estado, lo que dispara otro render, y así hasta que **React** entra en un bucle infinito y crashea.

El resultado es ese mensaje rojo desalentador: ***too many re-renders.***

> **¿Qué significa el error too many re-renders en React?** Significa que estás ejecutando una función que actualiza el estado durante el render, generando un bucle infinito. La solución es envolver la llamada en otra función que **React** solo ejecute cuando ocurra el evento.

La solución es envolver la función en una ***arrow function.***

```
<TodoItem onComplete={() => completeTodo(todo.text)} />
```

Ahora **React** recibe una función sin ejecutar. Solo le pone los paréntesis cuando el usuario hace clic. La aplicación deja de crashear y los *to-dos* se completan al instante, actualizando también el contador en pantalla.

Cómo eliminar ❌ un to do en React con el método splice

Eliminar sigue exactamente el mismo patrón que completar, pero en vez de modificar una propiedad, removemos el elemento del *array*. Creamos una propiedad `onDelete` en el `TodoItem` y la asociamos al ícono de la X con un `onClick`.

La función `deleteTodo` reutiliza casi toda la lógica de `completeTodo`:

- Recibe el texto del *to-do* a eliminar.
- Crea una copia del *array* con *spread*.
- Encuentra el índice con `findIndex`.
- Usa el método `splice` para remover el elemento.
- Actualiza el estado con `setTodos`.

```
const deleteTodo = (text) => {
        const newTodos = [...todos];
        const todoIndex = newTodos.findIndex( (todo) => todo.text === text );
        newTodos.splice(todoIndex, 1);
        setTodos(newTodos);
};
```

Qué hace el método splice y cómo funciona la analogía de las rodajas de pan

El método `splice` recibe dos parámetros principales: la posición donde "poner el cuchillo" y "cuántas tajadas cortar". En nuestro caso, el cuchillo va en `todoIndex` y cortamos solo una "rodaja", es decir, un *to-do*.

> **¿Cuál es la diferencia entre splice y delete en JavaScript?** `splice` modifica el *array* removiendo elementos y reorganizando los índices. `delete` deja un hueco con `undefined` en la posición. Para listas de *to-dos*, `splice` es más predecible.

Al recargar el navegador, ya puedes borrar ***to-dos*** completados y el contador refleja el cambio en tiempo real. La aplicación se siente viva.

Reto opcional para llevar tu to do counter al siguiente nivel

Un detalle que puedes mejorar es el mensaje del `TodoCounter` cuando todos los ***to-dos*** están completados. En vez de mostrar *Has completado 5 de 5 TODOs*, muestra algo más celebratorio como *Felicitaciones, completaste todos los TODOs* con un emoji.

```
import './TodoCounter.css';

function TodoCounter({completed, total}) {
    return (
      <h1 className="TodoCounter">
        {completed === total ? '¡Felicidades! Has completado todas tus tareas.' :
`Completaste ${completed} de ${total} TODOs`}
      </h1>
    );
  }

  export {TodoCounter};
```

🤖Iconos SVG en React con componentes

Tu app de ***to-dos*** en **React** ya reacciona a los clics, pero todavía se ve fea: una V para completar, una X para borrar, puro texto plano. Para llevarla a nivel portafolio necesitas **iconos SVG** bien implementados. Aquí verás cómo agregar iconos en **React**, ya sea con la librería ***React Icons*** o creando tu propia librería de iconos personalizados.

¿Cuándo conviene usar React Icons y cuándo crear tu propia librería?

La decisión depende de si tu equipo de diseño tiene iconos propios o si puedes apoyarte en sets ya existentes.

React Icons es la opción más rápida y popular. Importas el componente, lo usas y listo. Incluye repertorios enormes:

- Ant Design, ideal si te gusta ese estilo limpio.
- Bootstrap y Font Awesome, clásicos infaltables.
- Octicons de GitHub y Material Design.

- Hasta los iconos de Visual Studio Code.

Ahora, si tu equipo diseñó SVGs propios, te toca usarlos. No puedes meter un icono ***random*** de otra librería porque rompe la identidad visual. Ese es el caso de la app de *to-dos*: el equipo de diseño entregó dos archivos, `check.svg` y `delete.svg`, y hay que integrarlos creando una librería personalizada.

> **¿Qué es React Icons?** Es una librería que centraliza iconos de múltiples sets como Material Design, Font Awesome o Bootstrap, y los expone como componentes React listos para importar.

¿Cómo importar SVG en React con Create React App?

Create React App incluye soporte nativo para importar SVGs como componentes. La sintaxis es muy específica y hay que respetarla.

Primero arrastras tus archivos `.svg` a la carpeta `src`. Importante: si tu SVG tiene la propiedad `fill` con un color fijo, quítasela. Así puedes pasarle el color dinámicamente desde **React** más adelante.

Luego importas con esta sintaxis:

```
import { ReactComponent as CheckSVG } from './check.svg';
import { ReactComponent as DeleteSVG } from './delete.svg';
```

El truco está en `ReactComponent`. Así lo diseñaron los creadores de **Create React App.** Cuando importas varios SVGs en el mismo archivo, usas ***exports nombrados*** renombrando cada uno (`CheckSVG`, `DeleteSVG`) para evitar conflictos.

¿Cómo crear componentes de iconos reutilizables en React?

La idea es centralizar la lógica en un solo componente, `TodoIcon`, y que `CompleteIcon` y `DeleteIcon` solo le pasen propiedades.

Estructura de los componentes hijos

Cada icono específico delega el trabajo pesado a `TodoIcon` y solo le indica qué tipo y color quiere:

```
import React from 'react';
import { TodoIcon } from './TodoIcon';

function CompleteIcon() {
    return <TodoIcon type="check" color="gray" />;
}

export { CompleteIcon };
```

Lo mismo aplica para `DeleteIcon`, pero con `type="delete"`. Nombrar los componentes por su funcionalidad (`CompleteIcon`, `DeleteIcon`) en lugar de su forma (`XIcon`, `VIcon`) es buena práctica: el nombre describe qué hace, no cómo se ve.

Clases dinámicas con className

Dentro de `TodoIcon`, usas un `span` con `className` dinámico para que el CSS posicione cada icono correctamente:

```
<span className={`Icon-svg Icon-svg--${type}`}> {/* SVG aquí */} </span>
```

Si recibes `type="delete"`, se renderiza la clase `Icon-svg--delete`. Si recibes `check`, se renderiza `Icon-svg--check`. Cada clase aplica una posición distinta: arriba a la derecha o a la izquierda del *to-do*.

¿Cómo renderizar varios SVG según una prop sin condicionales complejos?

Usar un *if* o un ternario funciona con dos opciones, pero se vuelve ilegible si agregas más iconos. La solución elegante es un objeto que mapea tipos a componentes.

```
const iconTypes = {
    check: <CheckSVG className="Icon-svg" fill="gray" />,
    delete: <DeleteSVG className="Icon-svg" fill="gray" />,
};

function TodoIcon({ type }) {
    return (
        <span className={`Icon-svg Icon-svg--${type}`}> {iconTypes[type]} </span>
    );
}
```

Una línea por cada tipo de icono. Si mañana agregas un tercer SVG, solo sumas otra entrada al objeto. Sin *if*, sin *switch*, sin spaghetti.

> **¿Cómo cambiar el color de un SVG en React?** Quita el atributo `fill` del archivo SVG original y pásalo como **prop** al componente importado. Así el color se vuelve dinámico y reutilizable.

> **¿Por qué usar un objeto en lugar de if/else para renderizar componentes?** Porque escala mejor: agregar un nuevo caso es una línea, no un bloque condicional, y el código queda más legible.

Habilidades y conceptos que se trabajaron en la clase

Más allá del paso a paso, hay ideas que vale la pena retener para tus próximos proyectos.

- **Importación de SVG como componentes** con `ReactComponent` de **Create React App,** una sintaxis exclusiva de esa herramienta.
- **Exports nombrados con alias** (`as CheckSVG`) para evitar colisiones cuando importas archivos con el mismo nombre exportado.
- **Composición de componentes**: `CompleteIcon` y `DeleteIcon` son envoltorios delgados sobre `TodoIcon`, lo que centraliza la lógica visual.
- **Props como configuración**: pasar `type` y `color` permite que un mismo componente cambie de comportamiento sin duplicar código.

- **className dinámico con template literals**: combinar clases fijas con valores de **props** dentro de comillas invertidas.
- **Patrón de objeto-mapa para condicionales**: una alternativa limpia al `if/else` cuando tienes múltiples casos paralelos, técnica vista también en el Curso Práctico de JavaScript .
- **Limpieza de SVGs antes de importarlos**: quitar `fill` para controlar el color desde **React**.

Colores dinámicos en SVGs con React

Trabajar con iconos SVG en React abre la puerta a estilos dinámicos que reaccionan al estado de tu aplicación. Aquí aprendes **cómo pasar colores como props a un SVG**, organizar tus estilos por archivo y resolver el clásico problema del ***prop drilling*** al conectar eventos entre componentes.

Cómo separar estilos para el contenedor y el SVG

Cuando un icono deja de ser solo texto y pasa a ser un SVG, necesitas estilizar dos cosas: el contenedor y el gráfico en sí.

La solución es dividir las clases. Por un lado, una clase `icon-container` para la etiqueta `span` que envuelve el icono. Por otro, una clase `icon-svg` que aplica directamente sobre el SVG y le da dimensiones concretas.

- `width: 24px` para el ancho del icono.
- `height: 24px` para mantener la proporción cuadrada que pide el sistema de diseño basado en ocho.
- Clase aplicada vía prop `className` desde React, no escribiendo el atributo en el SVG manualmente.

En **React** no necesitas tocar el archivo SVG para añadir clases. Cuando importas el componente con su nombre asignado, puedes pasarle **props** como `className` y **React** las traduce automáticamente al elemento SVG renderizado.

> **¿Cómo aplico una clase CSS a un SVG importado en React?** Pasas la clase como **prop** `className` al componente SVG importado. **React** la inyecta directamente en el elemento al renderizar.

Por qué mover los estilos a su propio archivo

Mantener el CSS junto al componente que lo usa es una cuestión de orden, no de funcionalidad.

Si los estilos del icono viven en `TodoItem.css` pero ahora existe un componente `TodoIcon` con su propia lógica, lo natural es crear un `TodoIcon.css` y cortar esos estilos hacia ahí. Sigue funcionando igual, pero tu proyecto queda más fácil de mantener cuando crezca.

Cómo paso un color como prop a un componente SVG

La propiedad `fill` es el equivalente a `color` para los SVG, y puedes controlarla desde JavaScript.

La técnica consiste en convertir cada elemento de tu objeto `iconTypes` en una *arrow function* que recibe un parámetro `color`. Esa función retorna el **JSX** del SVG con el atributo `fill={color}` ya inyectado.

```
const iconTypes = {
    check: (color) => <CheckSVG className="icon-svg" fill={color} />,
    delete: (color) => <DeleteSVG className="icon-svg" fill={color} />,
};
```

Desde el componente `TodoIcon` invocas la función pasando el color recibido por **props**: `iconTypes[type](color)`. Así el mismo SVG puede pintarse en verde, gris o rojo según el contexto.

Cómo decidir el color según el estado del todo

La lógica de color vive en los componentes específicos `CompleteIcon` y `DeleteIcon`, no en `TodoIcon`.

- En `CompleteIcon` recibes la prop `completed`. Si es `true`, envías `color="green"`; si es `false`, envías `color="gray"`.
- En `DeleteIcon` el color por defecto es `gray`, porque el rojo solo aparece en *hover*.
- `TodoIcon` se mantiene agnóstico: solo recibe un color y lo aplica.

Cómo logro el efecto hover en un SVG con CSS

El ***hover*** no se resuelve con **React**, se resuelve con CSS apuntando a la propiedad `fill` del SVG hijo.

Desde `TodoIcon.css` usas selectores que detectan el contenedor y aplican estilo al SVG anidado:

```
.icon-container-delete:hover .icon-svg {
    fill: red;
}
.icon-container-check:hover .icon-svg {
    fill: green;
}
```

Esto deja claro algo poderoso: los SVG aceptan `fill` también desde CSS, no solo desde atributos HTML.

> **¿Puedo cambiar el color de un SVG con CSS?** Sí, usando la propiedad `fill` en una regla CSS que apunte al elemento `o a sus hijos`.

Cómo conecto eventos onClick a través de varios componentes

Los eventos en **React** solo funcionan sobre elementos del **DOM**, no sobre componentes personalizados.

Eso significa que si pones `onClick` en `, ese click no hace nada hasta que la prop viaje hasta un elemento real`

1. `TodoItem` envía `onComplete` al `CompleteIcon` y `onDelete` al `DeleteIcon`.
2. Cada icono recibe su evento y lo reenvía a `TodoIcon` como una **prop** genérica `onClick`.
3. `TodoIcon` aplica `onClick` directamente sobre el contenedor del SVG.

Esta traducción de nombres específicos (`onComplete`, `onDelete`) a uno genérico (`onClick`) mantiene a `TodoIcon` reutilizable para cualquier tipo de acción.

Qué es el prop drilling y por qué importa

Enviar la misma prop a través de tres o cuatro componentes solo para que llegue al destino final tiene nombre: ***prop drilling***.

Es funcional, pero engorroso. Un descuido tan simple como pasar `onComplete` donde debía ir `onDelete` rompe la funcionalidad sin lanzar ningún error visible. Es justo lo que ocurre cuando el botón de eliminar deja de responder mientras completar sí funciona.

> **¿Qué es prop drilling en React?** Es el patrón de pasar props a través de múltiples componentes intermedios que no las usan, solo para que lleguen a un componente profundo que sí las necesita.

```
{/* TodoIcon.js */}
import { ReactComponent as CheckSVG } from './check.svg';
import { ReactComponent as DeleteSVG } from './delete.svg';
import './TodoIcon.css';
const iconTypes = {
 "check": (color) => <CheckSVG className="Icon-svg" fill={color} />,
  "delete": (color) => <DeleteSVG className="Icon-svg" fill={color} />,
};
function TodoIcon({ type, color, onClick }) {
    return (
        <span className={`Icon-container Icon-container-${type}`} onClick=
{onClick}>
        {iconTypes[type](color)} </span>
    );
}
export { TodoIcon };
```

▶ TodoIcon.css

```
{/*DeleteIcon.js*/}
import React from 'react';
import { TodoIcon } from './';
function DeleteIcon({onDelete}) {
    return <TodoIcon type="delete" color="gray" onClick={onDelete} />;
}
export { DeleteIcon };
```

```
{/* TodoIcon.js */}
import React from 'react';
import { TodoIcon } from './';
function CompleteIcon({ completed, onComplete }) {
    return <TodoIcon type="check" color={completed ? '#4caf50' : 'gray'} onClick=
{onComplete} />;
}
export { CompleteIcon };
```

Persistencia de datos con localStorage en React

Aprende a implementar **localStorage** en una aplicación **JavaScript** para que los datos sobrevivan al cierre del navegador. Esta guía explica cómo guardar, leer y eliminar información persistente, transformando arrays y objetos con **JSON** para que tu app recuerde el estado del usuario.

Qué es localStorage y para qué sirve

localStorage es una API del navegador que permite persistencia de datos incluso si el usuario cierra la pestaña, la ventana o apaga el computador. Cuando vuelve a abrir tu aplicación, la información sigue ahí.

En una app de *todos*, esto evita que la lista vuelva al array por defecto cada vez que se recarga. Cada cambio se guarda tanto en el estado de **React** como en **localStorage**, y al cargar la app, el estado inicial se sincroniza con lo que ya estaba guardado.

¿Qué es **localStorage**? Es una API del navegador que guarda ***pares clave-valor como strings***, manteniéndolos disponibles aunque el usuario cierre el navegador o reinicie su equipo.

Cómo leer, guardar y eliminar datos en localStorage

La API expone tres métodos principales que vas a usar todo el tiempo. No funciona como una asignación normal de JavaScript: necesitas métodos específicos.

localStorage.getItem('clave'): lee el valor guardado bajo esa clave.
localStorage.setItem('clave', 'valor'): guarda un string bajo esa clave.
localStorage.removeItem('clave'): elimina ese item del almacenamiento.
Si escribes localStorage en la consola, ves todo lo que hay guardado actualmente. Y aquí viene lo interesante: cualquier cosa que guardes tiene que ser un string. Si intentas guardar un array directamente, te va a aparecer el famoso "[object Object]" y nada va a funcionar.

Por qué necesitas JSON.stringify y JSON.parse

Como **localStorage** solo acepta strings, necesitas dos herramientas para convertir estructuras de JavaScript:

JSON.stringify(valor): convierte un array u objeto en un string que sí se puede guardar.
JSON.parse(string): convierte ese string de vuelta a un array u objeto utilizable.
Si guardas un array de cinco elementos sin **stringify**, al leerlo vas a obtener algo de 250 caracteres en lugar de cinco items. La regla es simple: **stringify** al guardar, **parse** al leer.

> ¿Por qué localStorage solo guarda strings? Porque su especificación lo limita a pares clave-valor de tipo string. Para guardar arrays u objetos necesitas serializarlos con JSON.stringify y deserializarlos con JSON.parse.

Cómo conectar localStorage con el estado inicial de React

El flujo correcto en una app React es leer **localStorage** primero y usar ese valor como estado inicial. Si no existe, asignar un valor por defecto seguro.

```
const localStorageTodos = localStorage.getItem('TODOS_V1');
let parsedTodos;
if (!localStorageTodos) {
    localStorage.setItem('TODOS_V1', JSON.stringify([]));
    parsedTodos = [];
} else {
    parsedTodos = JSON.parse(localStorageTodos);
}

const [todos, setTodos] = React.useState(parsedTodos);
```

Este patrón usa la metodología ***error first***: programa primero el caso menos ideal. Si **localStorage** devuelve null, undefined o cualquier valor falsy, inicializas con un array vacío y lo guardas también en localStorage. Así **React** nunca recibe algo que no pueda iterar.

Fíjate en el detalle del nombre de la clave: usar TODOS_V1 con sufijo de versión te permite migrar la estructura de datos en el futuro sin romper la app de los usuarios actuales.

Cómo actualizar estado y localStorage al mismo tiempo

Cada vez que el usuario completa o elimina un todo, necesitas actualizar dos lugares: el estado de React y localStorage. En lugar de duplicar esa lógica, encapsúlala en una función única.

```
const saveTodos = (newTodos) => {
    const stringifiedTodos = JSON.stringify(newTodos);
    localStorage.setItem('TODOS_V1', stringifiedTodos);
    setTodos(newTodos);
};
```

Luego reemplazas todas las llamadas a setTodos por saveTodos dentro de funciones como completeTodo o deleteTodo. Con eso, cualquier cambio se refleja tanto en pantalla como en almacenamiento persistente.

Errores comunes al trabajar con localStorage

Durante el desarrollo es muy frecuente romper algo, y la causa casi siempre es la misma: datos mal formateados que ya quedaron guardados.

Guardar sin **stringify**: aparece "[object Object]" y **JSON.parse** falla con Unexpected token o in **JSON**.
Olvidar parsear al leer: el estado recibe un **string** en lugar de un **array** y se rompe .**filter** o .**map**.
No manejar el caso null: la primera vez que un usuario entra, no existe la clave y la app se rompe si no hay valor por defecto.
La solución rápida cuando algo se ensucia es ir a la consola y ejecutar **localStorage.removeItem**('TODOS_V1') para empezar limpio.

Por qué conviene mover esta lógica a un custom hook

Después de implementar todo esto, el componente principal queda con demasiada responsabilidad: maneja estado, lógica de negocio y persistencia. Esa mezcla hace que el código sea más difícil de leer y mantener.

La siguiente evolución natural es extraer toda la lógica de **localStorage** a un **custom hook** reutilizable, algo así como **useLocalStorage**. Eso permite limpiar el componente, reutilizar la persistencia en otras partes de la app y separar claramente las preocupaciones.

Custom Hook para Local Storage en React

Aprende a construir un ***Custom Hook*** en React llamado **useLocalStorage** para abstraer la lógica de persistencia de datos y mantener tu componente App limpio, reutilizable y fácil de mantener.

Cuando mezclas la lógica de tus todos con llamadas a **localStorage**, validaciones de **JSON.parse, JSON.stringify** y funciones **actualizadoras**, tu componente se vuelve difícil de leer. La solución elegante es mover toda esa lógica a un **hook** propio que cualquier componente pueda consumir.

¿Qué es un Custom Hook en React y por qué empieza con use?

Un ***Custom Hook*** es una función de JavaScript que encapsula **lógica reutilizable** basada en los hooks oficiales de React, como useState, useEffect, useContext o useCallback.

La convención que React te pide seguir, de forma opcionalmente obligatoria, es que **el nombre empiece siempre con la palabra use**. Así, tanto tú como cualquier persona que lea tu código sabe inmediatamente qué función es un hook y cuál es una función normal de la aplicación.

> ***¿Qué es un Custom Hook?*** Es una función que empieza con use y agrupa lógica reutilizable de React, como estados o efectos, para que la consumas desde cualquier componente sin repetir código.

¿Cómo abstraer Local Storage en un Custom Hook reutilizable?

La idea es que tu componente App no llame directamente a **localStorage**. Toda la lógica de leer, escribir, parsear y actualizar se mueve al hook **useLocalStorage**, y el componente solo consume el estado final.

Dentro del hook creas un estado interno con React.useState, pero en lugar de llamarlo todos lo llamas item, porque Local Storage trata todo su contenido como items. Así puedes guardar cualquier cosa: todos, usuarios, configuraciones, lo que sea.

¿Qué parámetros recibe useLocalStorage?

Para que el hook sea verdaderamente reutilizable, recibe dos argumentos clave:

itemName: el nombre clave bajo el cual se guarda la información en Local Storage. Reemplaza el string fijo TODOS_V1.
initialValue: el estado inicial por defecto. Puede ser un array vacío, un string, un número, un booleano o lo que necesites.
Con estos dos parámetros, una misma función sirve para guardar todos, usuarios o cualquier otro tipo de dato sin reescribir lógica.

¿Qué debe retornar el Custom Hook?

El hook retorna un array con dos elementos: el estado actual del **item** y una función **saveItem** que actualiza tanto el estado de **React** como el contenido en **Local Storage**.

No retornas el **setItem** original de **useState**, porque ese solo actualizaría **React**. La función **saveItem** hace ambas cosas a la vez: persiste en Local Storage y refresca el estado interno.

```
function useLocalStorage(itemName, initialValue) {
    const localStorageItem = localStorage.getItem(itemName);
    let parsedItem;

if (!localStorageItem) {
    localStorage.setItem(itemName, JSON.stringify(initialValue));
    parsedItem = initialValue;
} else {
    parsedItem = JSON.parse(localStorageItem);
}

const [item, setItem] = React.useState(parsedItem);

const saveItem = (newItem) => {
    localStorage.setItem(itemName, JSON.stringify(newItem));
    setItem(newItem);
};

return [item, saveItem]; }
```

¿Cómo se consume el Custom Hook desde el componente App?

En lugar de llamar a **React.useState**, ahora llamas directamente a tu **hook**. Como retorna un array, puedes nombrar las variables como prefieras, igual que con **useState**.

```
const [todos, saveTodos] = useLocalStorage('TODOS_V1', []);
```

El *primer argumento* es el **nombre clave** en Local Storage; el segundo es el **estado inicial**. El resto de funciones como **completeTodo**, **deleteTodo** y **addTodo** siguen funcionando exactamente igual, porque siguen llamando a **saveTodos** por dentro.

¿Por qué retornar un array y no un objeto? Con un array puedes renombrar las variables al desestructurar. Con un objeto, estarías obligado a usar los nombres exactos de las propiedades.

¿Cuándo vale la pena crear un Custom Hook?

No existe una regla fija de "a partir de tantas líneas, crea un hook". Lo que tienes es criterio, y ese criterio se forma con práctica .

Algunas señales útiles para decidir:

La lógica de tu componente mezcla varios temas distintos y cuesta leerla.
Vas a reutilizar la misma lógica en otro componente o proyecto.
Quieres separar responsabilidades para que un cambio en Local Storage no afecte la lógica de negocio.
Abstraer demasiado pronto también te puede quitar tiempo. Si la lógica es simple y solo se usa una vez, quizá no valga la pena.

¿Qué ganas con esta abstracción?

La aplicación se ve exactamente igual para los usuarios. La persistencia funciona, los todos se completan y se eliminan como antes. Lo que mejora es tu developer experience: el componente App ahora solo contiene lógica de todos, y toda la persistencia vive en un archivo separado, fácil de mantener y modificar.

Si alguien de tu equipo necesita tocar algo de Local Storage, sabe exactamente dónde buscar. Y si necesita ajustar los todos, no se topa con código de persistencia que lo distraiga.

Cómo organizar carpetas de componentes React

Organizar carpetas y componentes en **React JS** marca la diferencia entre un proyecto que cualquiera puede leer y uno que asusta apenas se abre. La meta es que tu carpeta `src` deje de ser un cajón desordenado y se convierta en una estructura clara, donde cada pieza viva donde corresponde y los *imports* se sientan naturales.

¿Por qué importa la estructura de carpetas en un proyecto React?

Imagina que entra una persona nueva al equipo, una *recruiter* o tu próximo *manager* y abre tu repositorio. Lo primero que verá es `package.json` y luego tu carpeta `src`. Si ahí encuentra 20 archivos sueltos entre JavaScript, CSS y SVG, el mensaje que transmites es de descuido, aunque tu lógica funcione perfecto.

La limpieza en archivos y carpetas habla de ti como profesional. No siempre será la prioridad: a veces toca lanzar, corregir un bug urgente o agregar una *feature* esencial. Pero si tienes 5 o 10 minutos libres, dedicarlos a ordenar la anatomía del proyecto es una inversión que paga rápido.

> **¿Qué es la estructura de carpetas en React?** Es la forma en que organizas componentes, estilos, imágenes y lógica dentro de tu proyecto para que sea fácil de leer, escalar y mantener.

¿Cuáles son las formas de organizar componentes en React JS?

No existe una única filosofía. Hay opiniones fuertes sobre esto y cada equipo defiende la suya. Estas son las dos aproximaciones más comunes que vas a encontrar.

Organización por tipo de archivo

Esta estructura divide `src` en carpetas según la naturaleza de cada archivo:

- Una carpeta `components` para todos los componentes de React.
- Una carpeta `styles` o similar para los archivos CSS.
- Una carpeta `images` o `assets` para imágenes y SVGs.
- Una carpeta `utils` o `helpers` para JavaScript que no depende de React.

Funciona muy bien en proyectos pequeños. El problema aparece cuando tu carpeta `components` empieza a tener 100 archivos y el CSS de un componente vive lejos de su lógica. Si abres `CreateTodoButton.js` y necesita su CSS, su SVG y un par de *helpers*, vas a saltar entre tres carpetas distintas para tocar una sola pieza.

Organización por funcionalidad o componente

La alternativa es crear una carpeta por cada sección o pieza importante de la aplicación. Cada carpeta encapsula todo lo que ese componente necesita: el código de React, el JavaScript específico, el CSS, las imágenes y los SVGs.

Por ejemplo, para el botón de crear tareas creas una carpeta `CreateTodoButton` y adentro guardas:

- Un `index.js` con el código del componente.

- Un `CreateTodoButton.css` con sus estilos.
- Cualquier asset o helper que ese botón use exclusivamente.

Esta es la estructura preferida en proyectos que crecen, porque cada *feature* queda autocontenida y el ruido entre carpetas baja muchísimo.

¿Cómo refactorizar un componente a su propia carpeta sin romper los imports?

La parte que más duele al reorganizar es ajustar los *imports*. Aquí va el flujo limpio aplicado al `CreateTodoButton`.

Primero creas la carpeta `CreateTodoButton` dentro de `src`. Luego mueves el archivo del componente y lo renombras a `index.js`. Después arrastras el `CreateTodoButton.css` a esa misma carpeta. Visual Studio Code te preguntará si quieres mover el archivo y aceptas sin miedo.

Como el componente y su CSS ahora viven en la misma carpeta, el *import* del CSS dentro del componente no cambia. Y aquí viene lo interesante: el componente `App` que lo consumía tampoco necesita ajustarse. Antes importabas `./CreateTodoButton`, y como la nueva carpeta se llama igual y contiene un `index.js`, el *import* sigue resolviéndose solo.

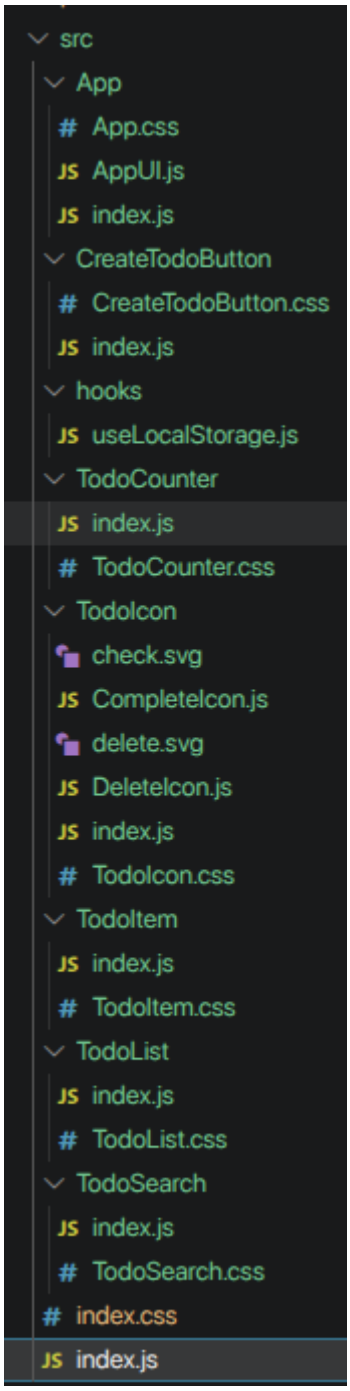
> **¿Por qué nombrar el archivo principal como index.js?** Porque cuando importas una carpeta, el *bundler* busca automáticamente un `index.js` adentro. Eso te permite escribir `import CreateTodoButton from './CreateTodoButton'` sin apuntar al archivo exacto.

La configuración de *transpilación* y empaquetado de Create React App también te permite omitir la extensión `.js` en los *imports*, así que el código queda más corto y legible.

¿Cómo verifico que la refactorización no rompió la aplicación?

Después de mover archivos, vuelve al navegador y recarga. Si tu botón sigue ahí, con sus estilos intactos y su comportamiento idéntico, la reorganización fue exitosa. Ese ciclo de mover, recargar y validar lo repites con cada componente que quieras encapsular.

La idea es replicar este patrón con el resto de la aplicación: cada pieza importante en su propia carpeta, con su `index.js` y todos los recursos que solo le pertenecen a ella.



Feature-First Directories en React

¿Cuál es la estrategia para organizar carpetas en un proyecto React?

Organizar las carpetas de tu proyecto puede ser fundamental para mantener un flujo de trabajo eficiente y claro. En el contexto de un proyecto React, la estructura de carpetas puede variar, pero hay enfoques que mejoran la manejabilidad del código. Una práctica común es crear una carpeta para cada componente, con un archivo `index.js` dentro, lo que facilita encontrar y gestionar cada componente.

¿Cómo se organiza un archivo de componente?

1. **Crear carpetas por componente:** Por cada componente, crea una nueva carpeta con el mismo nombre. Por ejemplo, para el componente `TodoCounter`, crea una carpeta llamada `TodoCounter`.
2. **Utiliza un archivo `index.js`:** Dentro de cada carpeta, nombra tu archivo de componente

como `index.js`. Así mantendrás una estructura clara y limpia.

3. **Agrupa los archivos relacionados:** Además del archivo `index.js`, coloca todos los archivos CSS y otros recursos (como imágenes o SVG) relacionados dentro de la misma carpeta.

¿Cuál es la ventaja de usar `index.js`?

El uso del archivo `index.js` es una práctica que ofrece dos ventajas clave:

- **Simplicidad en los imports:** Al utilizar `index.js`, cuando importas un componente, solo necesitas referenciar la carpeta, no el archivo específico. Esto reduce el riesgo de errores y simplifica el código.
- **Claridad organizativa:** Ofrece una manera clara y coherente de organizar tus componentes y recursos relacionados, asegurando que el código sea fácil de navegar y mantener.

¿Cómo se gestionan los errores al reorganizar las carpetas?

Es común encontrar errores al mover los componentes y sus archivos a nuevas localizaciones. Para lidiar con estos errores, sigue estos pasos:

¿Cómo ajustar los imports?

1. **Modificar las rutas de importación:** Al mover archivos, actualiza las rutas de importación según su nueva ubicación. Por ejemplo, si un componente ya no se encuentra en la misma carpeta, modifica la ruta añadiendo `../` para subir un directorio.
2. **Verificación en consola:** Después de hacer ajustes en las importaciones, revisa la consola para identificar y corregir cualquier error que pueda surgir.

¿Qué hacer con los warnings de ESLint?

A menudo, ESLint puede dar advertencias, como el uso de `==` en lugar de `===`. Estas advertencias:

- **No son errores:** No afectan la ejecución del código inmediatamente, pero es recomendable resolverlas.
- **Soluciones sencillas:** Sigue las sugerencias de ESLint para mantener un código limpio y libre de potenciales problemas.

¿Cómo se refactoriza un Custom Hook?

Refactorizar y extraer Custom Hooks puede mejorar la reutilización de código y la lectura del mismo.

¿Cómo se extrae un Custom Hook en un archivo separado?

1. **Crear el archivo del hook:** Si tienes un Custom Hook como `useLocalStorage`, crea un archivo `useLocalStorage.js`.
2. **Mover el código:** Corta y pega el código del hook al nuevo archivo, asegurando que React esté importado correctamente.
3. **Exportar el hook:** Utiliza exports nombrados para facilitar su importación en múltiples componentes.
4. **Ajustar las importaciones:** Desde el componente donde se utilizaba el hook, cambia la

importación para señalar al nuevo archivo del hook.

¿Por qué extraer Custom Hooks?

Extraer Custom Hooks tiene varias ventajas:

- **Reutilización:** Permite usar la funcionalidad del hook en diferentes componentes sin duplicar código.
- **Claridad de código:** Facilita la lectura y el mantenimiento del código al separar lógica compleja en archivos dedicados.
- **Escalabilidad:** Preparación para futuros cambios o ampliaciones sin afectar directamente a los componentes donde se usa el hook.

Tips para naming y abstracción de componentes React

En el mundo del desarrollo web, especialmente al trabajar con **React.js,** una de las preguntas que puede generarse con frecuencia es acerca de la organización de archivos y carpetas en un proyecto. La elección de nombrar todos los archivos como **index.js** puede conducir a algunas confusiones, pero también posee sus ventajas cuando se utiliza un editor de código como Visual Studio Code que ofrece claridad y facilita la navegación. A continuación, profundizaremos sobre esta metodología de organización y cómo puede afectar la claridad y la eficiencia en el desarrollo de un proyecto con **React.js.**

¿por qué nombrar los archivos como index.js?

La práctica de nombrar archivos como **index.js** se ha vuelto común en el ecosistema de **React** por varias razones. En primer lugar, ayuda a mantener una estructura de carpetas limpia y fácil de entender, donde cada carpeta representa un componente y el archivo **index.js** es el punto de entrada de ese componente.

herramientas que facilitan trabajar con múltiples index.js

Visual Studio Code y la mayoría de los editores modernos ofrecen funcionalidades que simplifican la labor cuando se tienen varios archivos con el mismo nombre en un proyecto:

Breadcrumbs: Muestra la ruta del archivo actual, facilitando saber en qué carpeta te encuentras.
Tabs con contexto: Al abrir varios archivos index.js, el editor puede mostrar el nombre de la carpeta para diferenciarlos.
Estas características permiten que, a pesar de tener muchos archivos llamados igual, la navegación y la distinción entre ellos sean claras y directas.

cómo facilitan los procesos de importación y versión de control

Tener archivos **index.js** simplifica también la declaración de imports en **React.js**. Puedes importar el componente solo con el nombre de la carpeta, omitiendo el nombre del archivo. Además, los sistemas de versión de control, como Git, pueden manejar de manera eficiente estos nombres de archivo ya que están asociados a carpetas específicas.

ventajas de un enfoque modular

Organizar los componentes en carpetas individuales y utilizar un enfoque modular ofrece varios beneficios:

Claridad: Cada componente está aislado en su carpeta, lo cual hace más evidente su propósito.
Mantenibilidad: Es más sencillo actualizar o refactorizar un componente específico sin causar efectos colaterales.
Reusabilidad: Los componentes pueden reutilizarse fácilmente en diferentes partes de la aplicación o en proyectos futuros.
separación de lógica y presentación

Una estrategia común en **React.js** es separar los componentes en stateful (con estado) y stateless (sin estado), distribuyendo la lógica de la aplicación y la presentación. Esto se hace creando componentes que manejan lógica y estado **(App.js)** y otros que se encargan principalmente de la presentación **(AppUI.js).** Al separar responsabilidades se obtiene un código más limpio y modular.

```
{/*App.js*/}
import React from 'react';
import './App.css';
import { useLocalStorage } from '../hooks/useLocalStorage';
import { AppUI } from './AppUI';

function App() {

  const [searchValue, setSearchValue] = React.useState('');
  const { item: todos, saveItem: saveTodos, loading, error } =
useLocalStorage('TODOS_V1', []);

  let completedTodos = todos.filter(todo => !!todo.completed).length;
  let totalTodos = todos.length;
  const searchedTodos = todos.filter(todo =>
todo.text.toLowerCase().includes(searchValue.toLowerCase()));

  const completeTodo = (text) => {
    const newTodos = [...todos];
    const todoIndex = newTodos.findIndex(todo => todo.text === text);
    newTodos[todoIndex].completed = !newTodos[todoIndex].completed;
    saveTodos(newTodos);
  }

  const deleteTodo = (text) => {
    const newTodos = [...todos];
    const todoIndex = newTodos.findIndex(todo => todo.text === text);
    newTodos.splice(todoIndex, 1);
    saveTodos(newTodos);
  }

  return (
    <AppUI
      loading={loading}
      error={error}
      totalTodos={totalTodos}
```

```
      completedTodos={completedTodos}
      searchValue={searchValue}
      setSearchValue={setSearchValue}
      searchedTodos={searchedTodos}
      completeTodo={completeTodo}
      deleteTodo={deleteTodo}
    />
  );
}

export default App;
```

```
{/*AppUI.js*/}
import React from "react";
import { TodoCounter } from '../TodoCounter';
import { TodoSearch } from '../TodoSearch';
import { TodoList } from '../TodoList';
import { TodoItem } from '../TodoItem';
import { CreateTodoButton } from '../CreateTodoButton';

function AppUI({
  loading,
  error,
  totalTodos,
  completedTodos,
  searchValue,
  setSearchValue,
  searchedTodos,
  completeTodo,
  deleteTodo,
}) {
  return (
    <div className="App">
      <TodoCounter completed={completedTodos} total={totalTodos} />
      <TodoSearch searchValue={searchValue} setSearchValue={setSearchValue} />

      <TodoList>
        {loading && <p>Cargando...</p>}
        {error && <p>Ocurrió un error</p>}
        {(!loading && !searchedTodos.length) && <p>Crea tu primer TODO</p>}
        {searchedTodos.map(tarea=>(
          <TodoItem
            key={tarea.text}
            text={tarea.text}
            completed= {tarea.completed}
            onComplete={()=>completeTodo(tarea.text)}
            onDelete={()=>deleteTodo(tarea.text)}
          />
        ))}
      </TodoList>

      <CreateTodoButton />
    </div>
  );
```

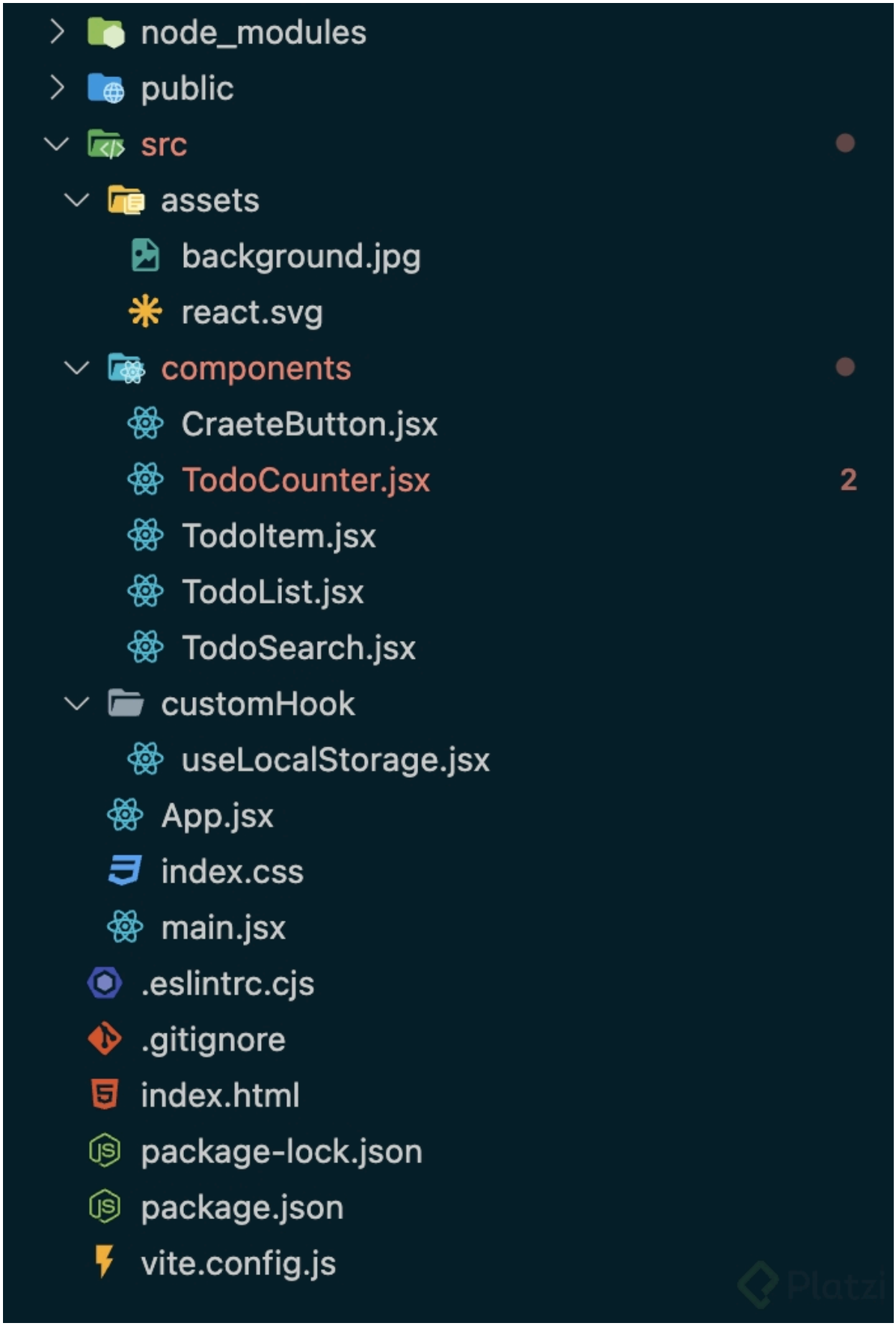
```
}

export { AppUI };
```

buenas prácticas y su impacto en equipos de trabajo

Mantener una estructura de archivos y carpetas organizada y profesional es crucial en el desarrollo web. Presentar un proyecto bien estructurado no solo facilita la lectura y el mantenimiento del código, sino que también impacta positivamente en los reclutadores y otros desarrolladores que puedan revisar tu trabajo.

Otra configuración de carpetas que sugirieron en el curso



useEffect para controlar renders costosos

El hook ***useEffect*** en **React** permite encapsular la lógica pesada o asíncrona de un componente para que solo se ejecute cuando realmente se necesita, evitando que cada render dispare llamadas costosas a una API o cálculos lentos. Es una herramienta clave para cualquier desarrollador frontend que busque optimizar la experiencia de usuario y el rendimiento de su aplicación.

Por qué necesitas useEffect en tus componentes de React

Cada vez que cambia un **estado**, **React** vuelve a renderizar el componente y reejecuta toda su lógica interna. El problema aparece cuando dentro de esa lógica hay procesos que tardan segundos, como una consulta a un ***backend*** o un cálculo pesado.

Sin protección, esos procesos se repiten en cada **render**, aunque solo deban correr una vez. Aquí entra **useEffect**, el **hook** que aísla esa parte costosa y la ejecuta bajo tus reglas.

> **¿Qué hace useEffect en React?** Encapsula lógica que no debe correr en cada render, como llamadas a una API o procesos asíncronos, y la ejecuta solo cuando tú decidas: una vez al inicio, o cuando cambien ciertos valores específicos.

Cómo se comporta useEffect frente al orden de ejecución

Un experimento sencillo con tres **console.log** lo deja claro. Si tienes Log 1, Log 2 y Log 3 en orden, pero envuelves el Log 2 dentro de un useEffect, el navegador imprime Log 1, Log 3 y al final Log 2.

Es decir, lo que vive dentro de un **useEffect** no respeta el orden lineal del código. React lo guarda y lo ejecuta después de procesar el resto de la lógica del componente.

Cuál es la sintaxis básica del hook

La estructura es directa: llamas a **React.useEffect** y le pasas una función como primer argumento. Dentro de esa función va la lógica que quieres proteger.

```
React.useEffect(() => { console.log('Log 2'); });
```

Con solo esto, el efecto corre después de cada render, pero al final del flujo, no en línea con los demás.

Para qué sirve el segundo argumento de useEffect

El segundo argumento es un *array* de dependencias y es la clave para controlar cuándo se ejecuta el efecto. Tienes tres escenarios posibles según lo que pongas dentro.

- Sin segundo argumento: el efecto se ejecuta después de cada render.
- Array vacío `[]`: el efecto se ejecuta una sola vez, en el primer render.
- Array con dependencias `[totalTodos]`: el efecto se ejecuta cuando cambia ese valor específico, sin importar qué otros estados se muevan.

Qué pasa con un array vacío

Al enviar `[]` como segundo argumento, el contenido del efecto corre solo en el primer render. Si después escribes en un input y cambia el estado searchedValue, el componente se vuelve a renderizar, pero el console.log dentro del useEffect no aparece de nuevo.

Eso es exactamente lo que necesitas para una llamada a una API que solo debe ocurrir al montar el componente.

Qué pasa cuando envías un estado al array

Si pasas un estado derivado como totalTodos dentro del *array*, el efecto se reactiva únicamente cuando ese valor cambia. Otros cambios de estado provocarán renders, pero no dispararán el efecto.

> **¿Cuándo se ejecuta useEffect con dependencias?** Solo cuando cambia alguno de los valores listados en el array. Si modificas otros estados del componente, el efecto se queda quieto.

Cómo aplicar useEffect a una llamada asíncrona real

Imagina un componente **TodoMessage** que muestra un mensaje obtenido desde una función **API** con `await`. Esa función es asíncrona y puede tardar cinco segundos en responder.

Sin **useEffect**, cada render dispararía una nueva llamada y el usuario vería la pantalla en blanco esperando. La solución combina tres piezas: un estado inicial vacío, un **useEffect** con array vacío y un *render* condicional.

- Inicializas el estado como un objeto vacío `{}`.
- En el JSX preguntas si existe la propiedad message; si no, muestras un texto de **cargando**.
- Dentro del **useEffect** haces el llamado a la API y, cuando responde, actualizas el estado con la información recibida.

De esta forma, el usuario ve cargando desde el primer instante, la **API** se consulta una sola vez, y cuando termina, el estado cambia y el componente muestra el mensaje real.

Cómo viaja un efecto por la fábrica de render

React tiene un *guardián de los efectos* que intercepta cada efecto antes de pasar por la fábrica de render. Lo aparta, deja que el resto de la lógica se ejecute, y después le da paso al efecto.

Cuando ese efecto actualiza el estado, se dispara un nuevo render. Pero como tiene un *array* vacío, el guardián lo reconoce y le dice que ya cumplió su trabajo. El componente se actualiza con la nueva información, sin volver a llamar a la API.

Qué ganas al usar useEffect correctamente

Dos beneficios claros cambian la calidad de tu aplicación.

- Mejor experiencia para el usuario, porque ve estados intermedios como cargando en lugar de pantallas vacías.
- Mejor rendimiento, porque los procesos pesados solo se ejecutan cuando son estrictamente necesarios.

Estados de carga y error

¿Cómo implementar estados de carga y error en una aplicación React usando hooks?

En las aplicaciones web actuales, los estados de carga y error son vitales para mejorar la experiencia del usuario, pues permiten informar al usuario sobre lo que ocurre en segundo plano. En este artículo, exploraremos cómo implementar estos estados de forma efectiva en una aplicación React utilizando un Custom Hook.

¿Cómo simular un retraso en el almacenamiento local?

Simular un retraso en la obtención de datos de `localStorage` nos ayuda a comprender cómo manejar datos asincrónicos en nuestra aplicación. Aunque `localStorage` suele ser rápido y síncrono, es relevante aprender a gestionar posibles demoras. Para lograr esto:

1. Usamos `setTimeout` para encapsular llamadas dentro de un retardo predefinido (por ejemplo, dos o tres segundos).
2. Devolvemos los datos simulando que provienen de una fuente asincrónica.

```
const useLocalStorage = (key, initialValue) => {
  const [storedValue, setStoredValue] = React.useState(initialValue);

  React.useEffect(() => {
    const retrieveData = async () => {
      await new Promise(resolve => setTimeout(resolve, 2000));
      const item = window.localStorage.getItem(key);
      item ? setStoredValue(JSON.parse(item)) : setStoredValue(initialValue);
    };
    retrieveData();
  }, [initialValue, key]);

  return [storedValue, setStoredValue];
};
```

¿Cómo manejar estados de carga y error?

Para mantener a los usuarios informados sobre la carga y posibles errores:

1. Creamos dos estados mediante el hook `useState`: `loading` y `error`.
2. `loading` inicializa a `true` y cambia a `false` cuando los datos han sido cargados.
3. `error` se inicializa en `false`, cambiando a `true` en caso de ocurrir un error.

```
const [loading, setLoading] = React.useState(true);
const [error, setError] = React.useState(false);

React.useEffect(() => {
  async function fetchData() {
    try {
      // Simular retraso
      await new Promise((resolve) => setTimeout(resolve, 2000));
      const item = window.localStorage.getItem(key);
      // Manejar datos
      item ? setStoredValue(JSON.parse(item)) : setStoredValue(initialValue);
```

```
    } catch (err) {
      setError(true);
    } finally {
      setLoading(false);
    }
  }
  fetchData();
}, [key, initialValue]);
```

¿Cómo mostrarlos en la interfaz?

Finalmente, mostramos mensajes adecuados según el estado:

- **Cargando:** Mostramos un mensaje simple como "Estamos cargando..."
- **Error:** Usamos un mensaje llamativo para indicar problemas, como "¡Hubo un error!"
- **Sin elementos:** Cuando el array está vacío, indicamos al usuario que puede empezar a añadir elementos.

```
return (
  <div>
    {loading && <p>Estamos cargando...</p>}
    {error && <p>¡Hubo un error!</p>}
    {!loading && !error && storedValue.length === 0 && <p>Crea tu primer
todo</p>}
    {!loading && !error && storedValue.map(todo => <Item key={todo.id} {...todo}
/>)}
  </div>
);
```

Recomendaciones finales

Cuando trabajamos con Custom Hooks que retornan múltiples valores, seguir las siguientes prácticas mejora la claridad de nuestro código:

- Retornar los valores dentro de un objeto en vez de un array para clarificar el significado de cada valor.
- Usar destructuración nombrada en el componente que consume el hook.

```
const { item, loading, error } = useLocalStorage('todos', []);
```

Manejar los estados de carga y error en **React** puede convertirse en una pesadilla si no proteges tu `useEffect` correctamente. Aquí verás cómo actualizar los estados de ***loading*** y ***error*** dentro de un custom hook con `localStorage`, envolver la lógica en un bloque *try/catch*, y evitar el temido bucle infinito que rompe tu aplicación.

Por qué mi estado nunca se actualiza desde localStorage

Al construir un **custom hook** para `localStorage`, es común crear los estados con `React.useState` y olvidar actualizarlos. Si defines `item`, `loading` y `error` con valores iniciales pero nunca llamas a `setItem`, `setLoading` o `setError`, tu aplicación se queda congelada en el estado inicial.

La solución dentro del efecto es directa: cuando termine la lectura de *localStorage*, llama a `setLoading(false)` para avisar que ya no estás cargando. Y si el valor parseado es distinto al *initial value*, llama a `setItem(parsedItem)` para reflejar los datos reales.

> **¿Por qué mi useState no refleja los datos de localStorage?** Porque el estado se queda con el *initial value* si nunca invocas el setter. Debes llamar a `setItem(parsedItem)` dentro del efecto una vez que hayas leído y parseado la información.

Cómo capturar errores con try catch

localStorage puede fallar en cualquier momento y no controlas cuándo. Por eso envuelves la lógica de lectura en un bloque `try` y, si algo sale mal, entras al `catch` para actualizar el estado de error.

Dentro del `catch` tienes dos opciones:

- Enviar el error tal cual con `setError(error)` si quieres saber el detalle.
- Enviar un booleano `setError(true)` si solo necesitas indicar que algo falló.

Si vas a mostrar el mensaje a usuarios finales, quédate con un genérico tipo Hubo un error. Y no olvides que cuando cae en el `catch` también debes hacer `setLoading(false)`, porque ya terminó el intento aunque haya fallado.

Cómo evitar el bucle infinito en useEffect

Una vez que agregas un `setTimeout` de 2000 milisegundos para simular una llamada asíncrona (como si consultaras una **API REST**), React deja de gritar con ***maximum update depth exceeded***. Pero eso no significa que tu código esté sano.

El efecto sigue siendo un bucle infinito silencioso: el estado inicial dispara el efecto, el efecto actualiza el estado con `setItem`, `setLoading` o `setError`, ese cambio provoca un nuevo render, el hook se vuelve a llamar y el ciclo empieza otra vez. React no lo marca como error porque el `setTimeout` lo hace ver natural, pero está mal.

> **¿Cómo hago que useEffect se ejecute solo una vez?** Debes pasarle un array de dependencias vacío como segundo argumento: `useEffect(() => {}, [])`. Son tres caracteres, un espacio incluido, que se olvidan facilísimo.

Qué son los ciclos de vida y por qué importa useEffect

Antes de los hooks, React trabajaba con clases que extendían de *React.Component* y usaban métodos de ciclo de vida como `componentDidMount` o `componentDidUpdate`. Con las versiones más recientes, `useEffect` se convirtió en la alternativa funcional a esos métodos.

Es cómodo, pero también engañoso: olvidar el array de dependencias es un error tan común como difícil de detectar cuando no salta ninguna alerta roja en la consola.

Cómo depurar el famoso undefined en filter

Otro error típico que aparece en esta implementación es intentar llamar a `.filter` sobre algo que no es un array. La consola dice claramente que no puede leer propiedades de *undefined*, y eso apunta a que el estado derivado (`searchedTodos`, por ejemplo) está recibiendo basura.

Cuando pasa esto, sigue este flujo de depuración:

1. Añade un `console.log` en el custom hook antes del `return` para ver qué valor tiene realmente `item`.

2. Revisa la línea exacta que apunta el error en el archivo `useLocalStorage`.
3. Verifica que los parámetros de la función estén bien declarados.

En este caso, el bug estaba en la firma del hook: por probar ejemplos, quedó escrito `useLocalStorage(itemName, [initialValue])` con corchetes alrededor de `initialValue`. Eso convierte el parámetro en un patrón de *array destructuring* y rompe el valor recibido. La corrección es dejarlo como un parámetro normal: `useLocalStorage(itemName, initialValue)`.

Cuándo borrar el localStorage en desarrollo

Si sospechas que el problema viene de datos corruptos guardados en el navegador, puedes borrar todo el *localStorage* desde la consola. En producción con usuarios reales esto sería catastrófico, pero en desarrollo es una herramienta válida para volver a un estado limpio y ejecutar los *default todos* que siempre funcionan.

Eso sí, si el error persiste después de limpiar, la falla no está en los datos sino en tu lógica, como pasó con el parámetro entre corchetes.

Cómo simular una carga asíncrona con setTimeout

Para imitar el comportamiento de una petición real y ver los estados de carga en acción, envuelve toda la lógica del `try/catch` dentro de un `setTimeout` con un delay de 2000 milisegundos. No uses `setInterval`, porque ese ejecutaría la función repetidamente.

La función que pasas como primer argumento contiene tu bloque `try/catch` completo, y el segundo argumento define cuántos milisegundos esperar antes de ejecutarla. Con esto, al recargar la aplicación verás por dos segundos el mensaje Estamos cargando antes de que aparezcan los TODOs, exactamente como pasaría con una API real.

```
{/*useLocalStorage.js*/}
import React from 'react';

function useLocalStorage(itemName, initialValue) {
  const [item, setItem] = React.useState(initialValue);
  const [loading, setLoading] = React.useState(true);
  const [error, setError] = React.useState(false);


  React.useEffect(() => {
    setTimeout(() => {
      try {
        const localStorageItem = localStorage.getItem(itemName);

        let parsedItem;

        if(!localStorageItem)
        {
          localStorage.setItem(itemName, JSON.stringify(initialValue));
          parsedItem=initialValue;
        }else{
          parsedItem=JSON.parse(localStorageItem);}
        setItem(parsedItem);
        setLoading(false);
      } catch (error) {
```

```
        setError(true);
        setLoading(false);
      }
    },2000);
  }, []);

  const saveItem = (newItem) => {
    const stringifiedItem = JSON.stringify(newItem);
    localStorage.setItem(itemName, stringifiedItem);
    setItem(newItem);
  }
  return {item, saveItem, loading, error};
}
export { useLocalStorage };
```

Loading Skeletons animados en React con CSS

Mostrar un texto plano de "cargando..." rompe la estética de cualquier aplicación. Crear **loading skeletons** en **React** te permite reemplazar ese mensaje aburrido por una versión animada y elegante del componente real, mejorando la experiencia de usuario mientras los datos se cargan desde **local storage** o una **API**.

La idea central es refactorizar la lógica de estados (carga, error, vacío) en componentes dedicados y aplicar CSS con gradientes animados para simular el contenido que está por aparecer.

Por qué refactorizar los estados en componentes dedicados

Cuando tu **TodoUI** mezcla validaciones de **loading**, **error** y listas vacías con párrafos sueltos, la maquetación se vuelve frágil. Lo más limpio es extraer cada estado a su propio componente.

En lugar de escribir el texto directamente dentro del render, puedes importar tres componentes específicos:

TodosLoading para el estado de carga.
TodosError para cuando algo falla.
EmptyTodos para cuando no hay tareas creadas.
Cada uno vive en su propia carpeta dentro de **src/**, con un **index.js** que exporta el componente y, opcionalmente, un archivo CSS asociado. Esta estructura te deja cambiar la interfaz de un estado sin tocar el resto de la aplicación.

¿Qué es un loading skeleton?

Es una versión visual y animada de un componente que imita su estructura final mientras se cargan los datos reales, en vez de mostrar un texto plano de carga.

Cómo replicar la estructura del TodoItem para el skeleton

El truco está en copiar la misma estructura HTML del componente real, pero vaciando su contenido. Si tu TodoItem usa un li con un ícono de completado, un párrafo de texto y un ícono de eliminar, tu **TodoLoading** debe tener exactamente esos tres bloques.

Las clases CSS sugeridas son:

loadingTodoContainer para el contenedor.
loadingTodoCompleteIcon para el círculo de la izquierda.
loadingTodoText para la barra central que simula el texto.
loadingTodoDeleteIcon para el ícono de la derecha.
Mantienes el mismo position: relative, display: flex, márgenes y sombras que el componente original, pero agregas un **border**-radius de 10 pixeles para suavizar los bordes y hacer evidente que es un placeholder.

Cómo animar el gradiente en CSS para simular carga

Aquí viene lo interesante. A todos los elementos del **skeleton**, excepto al texto, les aplicas un **background** con gradiente de colores. Para que la animación funcione, ese fondo necesita un tamaño exagerado, alrededor de 400% en alto y ancho.

Luego defines un @keyframes que mueve la posición del background en distintas direcciones, generando ese efecto de gradiente girando y parpadeando suavemente. El elemento permanece quieto en su lugar; lo que se mueve es el fondo gigante detrás de él.

```
@keyframes loadingAnimation {
  0% {
    background-position: 0% 50%;
  }
  50% {
    background-position: 100% 50%;
  }
  100% {
    background-position: 0% 50%;
  }
}

.loadingTodoContainer { background-size: 400% 400%; animation: loading-pulse 2s
infinite; border-radius: 10px; }
```

¿Cuántos skeletons debo mostrar?

Renderizar tres TodoLoading en lugar de uno solo ocupa más espacio en pantalla y comunica mejor que viene una lista, no un único elemento.

Cómo renderizar varios skeletons en App.js

En App.js, dentro del bloque que valida loading, llamas al componente **TodosLoading** tres veces seguidas. Visualmente da la sensación de una lista en construcción, lo cual es más fiel al resultado final que un solo placeholder.

Este detalle es estético, pero hace una diferencia notable en la percepción del usuario.

Qué otros componentes pueden tener su propio skeleton

El reto opcional es extender el patrón a más partes de la aplicación. Dos candidatos claros:

TodoSearch, el campo de búsqueda, que puede mostrarse como una barra animada antes de habilitarse.
TodoCounter, que durante la carga muestra "Has completado 0 de 0 todos" y luego salta al número real.
Un skeleton en el contador evita ese parpadeo de cifras, aunque dependiendo de tu criterio de UX puedes decidir dejarlo así si no afecta la experiencia.

Habilidades y conceptos que practicas con este reto

Refactor de estados condicionales hacia componentes independientes.
Creación de carpetas y index.js para exportar componentes en React.
Uso de gradientes CSS combinados con background-size exagerado.
Animaciones con @keyframes para mover el background-position.
Diseño consistente entre el componente real y su versión skeleton.

```
{/*TodosLoading.js*/}
import React from 'react';
import './TodosLoading.css';

function TodosLoading() {
  return (
    <div className="LoadingTodo-container">
      <span className="LoadingTodo-completeIcon"></span>
      <p className="LoadingTodo-text"></p>
      <span className="LoadingTodo-deleteIcon"></span>
    </div>
  );
}

export { TodosLoading };
```

¿Qué es React Context?

React Context resuelve uno de los dolores más comunes al construir aplicaciones con **React**: el famoso **prop drilling**. Si trabajas con componentes anidados en varios niveles, esta herramienta te permite crear un estado global y comunicar componentes sin pasar propiedades de padre a hijo a nieto.

Qué es el prop drilling y por qué te complica la vida

Imagina la estructura típica de una app de tareas. Tienes un componente **App** que llama a **TodoCounter**, **TodoSearch**, **TodoList** y **CreateTodoButton**. A su vez, **TodoList** llama a **TodoItem**. Hasta ahí todo bien, pero la comunicación entre ellos se hace con **props**, y eso se vuelve un laberinto.

El componente **App** envía propiedades a **TodoCounter**, otras distintas a **TodoSearch**, otras a **TodoList**, y este último vuelve a pasarlas al **TodoItem**. Cuando un componente actualiza el estado, ese cambio se propaga hacia arriba y luego baja a otros componentes que ni siquiera son sus hijos directos. Ese flujo enredado es el **prop drilling**, y en aplicaciones grandes como **Platzi**, **Google** o **GitHub** se vuelve insostenible.

> ¿Qué es el prop drilling? Es cuando una propiedad atraviesa múltiples niveles de componentes solo para llegar al que realmente la necesita, aunque los intermedios no la usen.

Cómo cambia React Context la forma de pensar la app

En lugar de ver tus componentes como un árbol con abuelos, padres e hijos, **React Context** te invita a verlos como piezas independientes que se conectan directamente al estado global. Cada componente le pide al contexto solo la información que necesita, sin importar en qué nivel esté.

Cómo identificar el problema en tu propio código

En una app real, el componente App suele traer datos desde un custom hook como **useLocalStorage**, crear **estados**, definir **efectos** y actualizadores, y luego entregar todo a un componente como **AppUI**. El costo de separar lógica e interfaz es que **AppUI** recibe un montón de **props** que no usa directamente, sino que reenvía a sus hijos.

Por ejemplo, **AppUI** recibe **completedTodos** y **totalTodos** solo para pasárselos a **TodoCounter**. Recibe **searchValue** y **setSearchValue** únicamente para entregárselos a **TodoSearch**. Esa cadena de reenvíos es el síntoma claro de que necesitas un contexto.

Qué propiedades sí pertenecen al componente y cuáles no

Dentro de **AppUI** realmente se usan **loading**, **error**, **searchedTodos**, **completeTodo** y **deleteTodo**. El resto solo está de paso. Ese filtro mental te ayuda a decidir qué mover al contexto: todo aquello que un componente reenvía sin usar.

Cómo crear un contexto con React.createContext

El primer paso es crear una carpeta llamada **TodoContext** dentro de **src** y un archivo **index.js**. Allí importas **React** y usas la herramienta **React.createContext**() para inicializar tu contexto.

```
import React from 'react';
const TodoContext = React.createContext();
```

Este contexto expone dos componentes internos: **TodoContext.Provider** y **TodoContext.Consumer**. El **Provider** entrega la información, el **Consumer** la recibe. Puedes usarlos directamente, pero la práctica más limpia es crear tu propio **Provider** personalizado.

> ¿Para qué sirve React.createContext? Crea un objeto de contexto que incluye un Provider y un Consumer, los dos componentes con los que compartes y consumes datos globales en tu app.

Cómo construir un TodoProvider personalizado

En vez de exportar **TodoContext.Provider** directamente, defines un componente **TodoProvider** que envuelve al **Provider** y encapsula toda la lógica que antes vivía en el componente **App**.

Mueves la llamada a **useLocalStorage** al **TodoProvider**.
Trasladas los **estados**, **estados derivados** y **actualizadores**.
Expones todo en la prop **value** del **Provider**.
Recibes **props.children** para insertar cualquier componente dentro.
Este patrón te da tres beneficios: un nombre más cómodo sin puntos intermedios, la posibilidad de exportarlo como un componente común, y un lugar único para centralizar la lógica compartida.

```
function TodoProvider({ children }) {
    // toda la lógica de estados y actualizadores
    return (
        <TodoContext.Provider value={{ loading, error, searchedTodos,
completeTodo, deleteTodo }}>
            {children}
        </TodoContext.Provider> );
}
```

Cómo consumir el contexto con TodoContext.Consumer

Una vez envuelves tu **AppUI** dentro del **TodoProvider** en el componente **App**, cualquier hijo puede acceder al **contexto**. Para leerlo usas TodoContext.Consumer, que sigue un patrón llamado **render props**.

Esto significa que el Consumer no espera componentes directos como hijos, sino una función que recibe el value del contexto y retorna los elementos a renderizar.

```
<TodoContext.Consumer>
    {({ loading, error, searchedTodos, completeTodo, deleteTodo }) => (
        <TodoList> {/* render de los todos */}
        </TodoList> )}
</TodoContext.Consumer>
```

Por qué se llaman render functions

La función que pasas como hijo al **Consumer** recibe como parámetro el objeto que guardaste en **value**. Desde ahí desestructuras solo lo que necesitas. Es la forma en que **React** conecta el **dato global** con el ***árbol de componentes*** sin romper la convención de **JSX**.

> ¿Cuándo usar Provider y cuándo Consumer? El Provider envuelve a los componentes que comparten el estado. El Consumer se usa dentro de cualquier componente hijo que quiera leer ese estado.

Qué pasa con TodoCounter y TodoSearch

Al quitar las props a **TodoCounter** y **TodoSearch**, esos componentes dejan de funcionar temporalmente. Buscan **totalTodos**, **completedTodos**, **searchValue** y **setSearchValue**, pero ya no los reciben. La solución es consumir el contexto desde adentro de cada uno.

El **TodoContext.Consumer** funciona y lo encontrarás en muchos proyectos profesionales de React, pero existe una forma más cómoda de consumir ese contexto sin anidar funciones dentro de funciones.

useContext

El manejo eficiente de los datos y el estado dentro de las aplicaciones **React** es una pieza fundamental para garantizar un desarrollo escalable y un rendimiento óptimo. Recientemente hemos avanzado al aprender sobre **React Context**, una herramienta diseñada para evitar el tedioso "**prop drilling**" o la necesidad de pasar **props** a través de varios niveles de componentes que no las utilizan directamente. Pero hoy, avanzaremos un paso más allá explorando el uso del **hook** `useContext`, que puede simplificar aún más la forma en que accedemos a nuestro contexto

en componentes React, haciendo nuestra vida como desarrolladores mucho más cómoda y, por qué no, también más estética.

¿Qué es el prop drilling y cómo puede ayudarnos React Context?

Para comprender cómo `useContext` mejora nuestro flujo de trabajo, es esencial entender primero el problema que aborda. El "**prop drilling**" se refiere a la práctica de pasar datos a través de una jerarquía de componentes en **React**, incluso a aquellos que no necesitan esos datos para su funcionamiento. Esto puede complicar el mantenimiento del código y dificultar el seguimiento del flujo de datos.

¿Cómo funciona `useContext` para acceder al contexto global?

El hook `useContext` nos permite suscribirnos a un contexto **React** dentro de un componente funcional. Basta con proporcionarle al hook el contexto deseado para que `useContext` haga "la magia" de compartir los datos necesarios directamente con el componente que los requiere.

- Importamos `useContext` desde React.
- Invocamos `useContext` y le pasamos el contexto que queremos consumir.
- Recibimos los datos necesarios directamente, sin necesidad de pasar **props** a través de la jerarquía de componentes.

¿Cómo simplifica `useContext` nuestro código?

En lugar de envolver nuestros componentes con un componente `Consumer` y usar una función render para acceder al contexto, `useContext` nos permite hacerlo en una sola línea de código. Esto resulta en un código más limpio y legible.

- Eliminamos el uso excesivo del componente `Consumer`.
- Reducimos la cantidad de código al no necesitar funciones render adicionales.
- Mejoramos la legibilidad y la estructura del código al evitar el "**prop drilling**".

¿Puede `useContext` reemplazar completamente al `Consumer`?

Si bien `useContext` simplifica y mejora la legibilidad del código, hay situaciones en las cuales las **render functions** y el componente `Consumer` pueden ser más adecuados. Por ejemplo, algunas complejidades de implementación o detalles específicos de rendimiento a veces hacen que estas técnicas sean preferibles.

- Los **hooks** no reemplazan completamente a las **render functions**.
- Es beneficioso para los desarrolladores estar familiarizados con ambas aproximaciones.
- La elección entre `useContext` y `Consumer` depende del caso de uso particular y las preferencias del desarrollador.

Próximos pasos en tu aprendizaje de React

Una vez que te sientas cómodo utilizando `useContext` para acceder y manejar el contexto global de tu aplicación **React**, el siguiente paso en tu viaje de aprendizaje es explorar otras potentes herramientas dentro del ecosistema **React**, como los **React Portals**.

- Continúa la práctica creando y consumiendo contextos con `useContext` y **render functions.**

- Profundiza en patrones de render y composición revisando recursos adicionales y cursos especializados.
- Desarrolla tus habilidades con React Portals, que te permitirán renderizar componentes fuera de la jerarquía de componentes principal.

En resumen, `useContext` es una herramienta que todo desarrollador React debería tener en su conjunto de herramientas para escribir un código más limpio, modular y fácil de mantener. Al mismo tiempo, recuerda que el conocimiento de diferentes métodos para manejar el contexto es invaluable en situaciones donde la complejidad del proyecto lo requiera.

¿Qué son los React Portals?

¿Qué son los React Portals y cómo funcionan?

React Portals es una poderosa herramienta que ofrece **React** para mover componentes entre diferentes nodos de HTML, casi como si de magia se tratara. Imagina que tienes un componente que deseas renderizar en un lugar completamente distinto, fuera del flujo DOM principal de tu aplicación. Los **Portals** te permiten hacer justamente eso: teletransportar componentes entre nodos, manteniendo la estructura del componente principal. Esto resulta especialmente útil cuando se maneja contenido dinámico como modales o tooltips, donde el **UI** necesita aparecer fuera del contenedor principal.

¿Cómo se implementan los Portals en un proyecto React?

Para implementar **Portals** en un proyecto **React** primero se debe crear un **nodo HTML** adicional en el archivo `index.html`. Este nodo será el destino al que se teletransportará el contenido del componente. La implementación típica requiere las siguientes acciones:

1. **Crear el nodo en HTML**: Añadir un div con un ID único al archivo `index.html`. Por ejemplo `<div id="modal"></div>`.
2. **Uso de `react-dom.createPortal`**: Dentro del componente, usar `react-dom.createPortal` para definir qué contenido será teletransportado y a qué nodo HTML.

```
import ReactDOM from 'react-dom';

function Modal({ children }) {
  return ReactDOM.createPortal(
    <div className="modal">
      {children}
    </div>,
    document.getElementById('modal')
  );
}
export { Modal };
```

¿Cómo controlar la visibilidad de un modal con estado?

Para controlar cuándo un modal debe estar visible se puede usar un **estado** en React, por ejemplo `openModal`. El modal solo debe renderizarse cuando este estado sea `true`. Se puede lograr lo siguiente:

- Crear un estado `openModal`:

```
const [openModal, setOpenModal] = React.useState(false);
```

- Renderizar condicionalmente el modal:

```
{openModal && <Modal>Contenido del modal</Modal>}
```

- Usar un botón o evento para alternar el estado `openModal`:

```
<button onClick={() => setOpenModal(!openModal)}>Toggle Modal</button>
```

¿Cómo hacer el estilo del modal más atractivo?

Un modal bien diseñado puede mejorar significativamente la experiencia del usuario. Aquí algunas sugerencias para estilizar un modal usando CSS:

- **Cobertura completa**: Hacer que el modal abarque toda la pantalla usando CSS con `position: fixed`, `top: 0`, `left: 0`, `width: 100%` y `height: 100%`.
- **Fondo translúcido**: Usar un color de fondo con transparencia para resaltar el contenido del modal y crear un efecto de desenfoque del fondo.
- **Centro del contenido**: Centrar el contenido del modal vertical y horizontalmente para mejor accesibilidad usando `flexbox`.

Ejemplo de CSS:

```
.modal {
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  background-color: rgba(0, 0, 0, 0.5);
  display: flex;
  justify-content: center;
  align-items: center;
}
```

Conclusión

Implementar Portals en React para crear modales y otros componentes de UI dinámicos es una táctica beneficiosa que maneja eficientemente la renderización de componentes fuera del DOM principal. Usar estados para controlar la visibilidad y CSS para el estilo asegura que estos elementos no solo funcionen bien, sino que también se vean bien.

Crea tu primer proyecto con Next.js

Next.js es un framework de **React.js** que combina ***server-side rendering***, enrutamiento propio, optimización de imágenes y de fuentes en una sola herramienta. Si vienes de **React** y quieres dar el siguiente paso hacia aplicaciones full stack, aquí descubres cómo arrancar tu primer proyecto y qué decisiones técnicas implica.

¿Qué es Next.js y por qué se volvió tan popular?

Next.js es un framework construido sobre **React.js**, respaldado por **Vercel** y mantenido por un equipo que prioriza la experiencia del desarrollador y del usuario final. Su CEO, **Guillermo Rauch**, se define como ***frontend engineer***, y ese enfoque se nota en cada decisión del producto.

Es el competidor más fuerte de Create React App en cuanto a adopción. Tan influyente se ha vuelto que muchas implementaciones nuevas de **React** aparecen primero en **Next.js**, lo que le da un peso enorme dentro del ecosistema.

> **¿Qué hace especial a Next.js frente a React puro?** Te entrega *server-side rendering*, router propio, optimización de imágenes y fuentes listos para usar, sin que tengas que configurarlos desde cero.

Eso sí, no todo es color de rosa. Algunas decisiones de arquitectura, integración o compatibilidad con herramientas externas no son del agrado de toda la comunidad. Es un **trade-off** que vale la pena conocer antes de comprometerte con el framework.

¿Cómo crear tu primer proyecto con Create Next App?

El equipo de **Next.js** creó una herramienta sospechosamente parecida a **Create React App**: **`create-next-app`.** Para arrancar con la última versión de todo, ejecuta:

```
npx create-next-app@latest next-test
```

A diferencia de Create React App, aquí te aparece un formulario interactivo con preguntas de configuración . Por ejemplo:

- ¿Quieres usar TypeScript?
- ¿Quieres usar ESLint?
- ¿Activamos otras herramientas opcionales?

Esto te permite tener un ***boilerplate*** mucho más afinado desde el segundo cero, en lugar de partir de una base genérica.

¿Qué archivos y carpetas trae el proyecto?

Al entrar al proyecto encuentras una estructura clara que define cómo trabaja **Next.js**:

- **pages/**: contiene `index.js`, `_app.js` y la carpeta `api`. Aquí vive el *file system routing*.
- **public/**: archivos estáticos del proyecto.
- **next.config.js**: configuración avanzada del framework.
- **package.json**: con apenas tres dependencias principales (`react`, `react-dom`, `next`).

Dentro del `package.json` también encuentras los scripts clave: `next dev` para desarrollo, `next build` para generar la versión de producción y `next start` para arrancar el servidor en producción.

¿Cómo funciona el enrutamiento por archivos en Next.js?

La carpeta `pages` no es decorativa: define las rutas de tu aplicación a partir de los nombres de los archivos. Esto se conoce como *file system routing* y elimina la necesidad de configurar un router manualmente.

Además, Next.js usa archivos especiales como `_app.js` para modificar el comportamiento global de la aplicación o de rutas específicas. Y trae componentes propios como `next/head` y `next/image`, pensados para resolver problemas comunes con SEO y rendimiento.

> **¿Qué es file system routing?** Es un sistema donde cada archivo dentro de `pages` se convierte automáticamente en una ruta de tu aplicación, sin configuración extra.

¿Por qué Next.js es un framework full stack?

Aquí viene lo interesante. Next.js no se queda en el front-end: te permite crear tus propias APIs dentro de la carpeta `pages/api`, igual que harías con Express, Koa o Fastify en Node.js.

Eso significa que en un mismo proyecto puedes:

- Construir la interfaz con React.
- Renderizar del lado del servidor con *server-side rendering*.
- Exponer endpoints de API consumidos por tu propio front-end.
- Desplegar todo como una aplicación unificada.

Esa es la verdadera definición de framework **full stack**: una herramienta que cubre el ciclo completo de desarrollo. ¿Siempre conviene? No. A veces un sitio estático con archivos exportados a GitHub Pages es más que suficiente. Otras veces necesitas la flexibilidad de un servidor Node.js corriendo en producción.

¿Cuándo usar server-side rendering y cuándo sitios estáticos?

Next.js soporta ambos modos de render y la elección depende del tipo de aplicación que construyas:

- **Sitios estáticos**: ideales para contenido que no cambia con frecuencia, despliegues simples y costos bajos.
- **Server-side rendering**: útil cuando necesitas datos frescos en cada request, personalización por usuario o SEO dinámico.

Aprender los dos enfoques te da criterio para decidir qué arquitectura usar en cada proyecto, y ese criterio vale más que memorizar comandos.

¿Qué pasa con las versiones y la migración entre ellas?

A diferencia de **Create React App,** que tiende a mantenerse estable, **Next.js** cambia rápido. Cada versión mayor (12, 13, y siguientes) introduce diferencias importantes en API, configuración o convenciones.

Si tomas un proyecto existente y necesitas migrarlo, el flujo es claro:

1. Actualiza la versión en el `package.json`.
2. Borra `node_modules` y el `package-lock.json`.
3. Reinstala dependencias.
4. Revisa la guía oficial de migración para aplicar los cambios necesarios.